

PBench - Benchmarking Relational Provenance Systems

[Experiment, Analysis & Benchmark]

Pengyuan Li
Illinois Institute of Technology
USA
pli26@hawk.illinoistech.edu

Davood Rafiei
University of Alberta
Canada
drafie@ualberta.ca

Boris Glavic
University of Illinois, Chicago
USA
bglavic@uic.edu

ABSTRACT

Data provenance is fundamental to establishing trust in data, debugging complex computations, explaining query outcomes, and supporting auditing and compliance. Over the past two decades, several relational provenance systems have been proposed, but comparing them remains difficult because they differ not only in runtime, but also in the provenance models they support, the SQL features they handle, the representation they produce, and the extent to which captured provenance can support downstream tasks. Existing evaluations typically focus on a subset of these dimensions, making it difficult to understand the practical trade-offs among competing designs. We present *PBench*, the first benchmark for relational provenance systems that evaluates these dimensions jointly. *PBench* combines three complementary workload families: (1) *SQL-centric workloads* that stress core relational operators and query structures, (2) *provenance-centric workloads* that vary provenance size and representation characteristics, and (3) *use-case-driven workloads* that evaluate end-to-end tasks such as reproducibility and debugging. Using *PBench*, we compare four representative provenance systems—GProM, ProvSQL, SQLProv, and SmokedDuck. Our results reveal clear trade-offs between capture cost, storage overhead, provenance expressiveness, and downstream utility.

PVLDB Reference Format:

Pengyuan Li, Davood Rafiei, and Boris Glavic. PBench - Benchmarking Relational Provenance Systems [Experiment, Analysis & Benchmark]. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/pli26/PBench_2027.git

1 INTRODUCTION

Relational data provenance has become a core component of modern data management, supporting tasks such as debugging query results, explaining analytical outcomes, maintaining derived data products, reproducing computations, and auditing data-intensive workflows. The database community has proposed a rich collection of provenance models, representations, and systems. Yet despite

this substantial progress, there is still no principled way to compare relational provenance systems. As a result, it remains unclear how different design choices affect performance, storage overhead, expressiveness, and the ability to support downstream applications.

A key challenge in comparing provenance systems lies in the diversity of their underlying models and architectures. Provenance describes how output data depends on input data and the transformations applied during query evaluation [15, 25, 29, 33]. At the tuple level, systems may capture lineage [18–20, 22], which records the input tuples contributing to each output tuple, or more expressive forms such as provenance polynomials [7, 59] that additionally encode how tuples are combined to derive a result. Systems also differ in how provenance is captured and stored, ranging from query rewriting approaches to database extensions and specialized execution engines. These choices affect SQL feature support, runtime overhead, storage requirements, and supported tasks.

EXAMPLE 1.1. Consider the database shown in Fig. 1 and the following query Q_{emp} that returns employees with their salary whose departments is located in the region “North”.

```
SELECT name, salary
FROM Dept JOIN Emp ON (did = deptid)
WHERE region = 'North';
```

Fig. 2 shows the provenance of each result of Q_{emp} according to the lineage [22] and provenance polynomials [28] models. Lineage captures which input tuples were used to derive a result, e.g., Alice (r_1) is based on the Emp tuple t_1 and the department tuple s_1 . Provenance polynomials make explicit how input tuples are combined. In this example, r_1 is the result of joining t_1 with s_1 , represented as multiplication.

Provenance is used to support a variety of tasks, including tracing erroneous results back to their sources (backward tracing), identifying outputs affected by input errors (forward tracing), and explaining complex query behavior. Beyond these user-facing applications, provenance also enables probabilistic query processing [61, 64], hypothetical reasoning (e.g., what-if and how-to queries) [13, 17, 21, 45], reproducibility of computational experiments [16, 62], and efficient maintenance of derived data products under updates [34]. As Ex. 1.1 illustrates, different provenance models vary in expressiveness. These differences have important practical implications as applications differ in what is the minimal expressiveness required to support them: less expressive models are insufficient while capturing too expressive provenance will potentially incur unnecessary runtime overhead and storage cost.

Despite the maturity of the field, relational provenance lacks a standardized benchmark. Existing systems are typically evaluated in isolation, often using workloads tailored to their particular provenance model, implementation strategy, or target application.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

eId	name	deptId	salary	$\mathbb{N}[X]$	dId	deptName	region	$\mathbb{N}[X]$
1	Alice	101	110,000	t_1	101	Sales	North	s_1
2	Peter	101	110,000	t_2	102	IT	North	s_2
3	Bob	102	80	t_3	103	Marketing	South	s_3
4	John	103	95,000	t_4	104	HR	East	s_4
5	George	104	155,000	t_5	105	Finance	North	s_5

Figure 1: Example employee database with relations Emp and Dept

As a result, published results are often difficult to compare, and fundamental questions remain unanswered: How does provenance overhead vary across SQL features? When is a more expressive provenance representation worth its additional cost? Do compact representations remain advantageous once downstream tasks such as debugging, tracing, or reproducibility are considered? Without a common benchmark, these trade-offs remain poorly understood. While prior provenance challenges [1, 49] have evaluated workflow and scientific provenance, they do not evaluate the aspects of provenance relational systems mentioned above.

Contributions. We introduce PBench, a benchmark for systematically evaluating relational provenance systems. PBench evaluates systems along three complementary dimensions: (i) SQL-centric workloads that stress relational operators, query structures, and feature coverage; (ii) provenance-centric workloads that vary provenance size, complexity, and representation characteristics; and (iii) use-case-driven workloads that evaluate end-to-end tasks such as debugging, tracing, and reproducibility. By jointly considering these dimensions, PBench enables evaluation beyond provenance-capture performance alone, exposing trade-offs among provenance expressiveness, runtime overhead, storage cost, and downstream usability. Using PBench, we conduct the first comprehensive experimental comparison of representative provenance systems, including SmokedDuck, ProvSQL, SQLProv, and GProM, on both PostgreSQL and DuckDB. Systems tightly integrated with the query execution engine achieve the lowest capture overhead, while systems optimized for provenance consumption can outperform lower-overhead alternatives when complete debugging or tracing workflows are considered. For certain query constructs such as nested subqueries, the ability to factorize provenance is critical for performance. We further show that highly expressive provenance models may incur substantial storage and runtime costs that are unnecessary for many applications, and that provenance-capture strategies, representation choices, and implementation choices often have great impact on end-to-end efficiency. These findings suggest that provenance systems should be designed and evaluated with their target use cases in mind rather than optimized for a single performance metric, and provide practical guidance for the design of future provenance-aware database systems.

The remainder of this paper is organized as follows. We introduce common provenance models in Sec. 2, discuss motivating use cases for database provenance and identify their requirements in Sec. 3, and give an overview of the evaluated systems and their design decisions in Sec. 4. We present our benchmark PBench in Sec. 5 and use this benchmark to evaluate the systems in Sec. 6. Related work is discussed in Sec. 7. We present conclusions and discuss future work in Sec. 8.

	name	salary	Lineage	$\mathbb{N}[X]$
r_1	Alice	110,000	$\{t_1, s_1\}$	$t_1 \cdot s_1$
r_2	Peter	110,000	$\{t_2, s_1\}$	$t_2 \cdot s_1$
r_3	Bob	80	$\{t_3, s_2\}$	$t_3 \cdot s_2$

Figure 2: Result of Q_{emp} with lineage and provenance polynomials

	region	Witness	Lineage	Polynomials ($\mathbb{N}[X]$)
r_1	North	$\{t_1, s_1\}$	$\{t_1, t_2, s_1\}$	$\delta(t_1 \cdot s_1 + t_2 \cdot s_1)$
r_2	East	$\{t_6, s_4\}$	$\{t_6, s_2\}$	$\delta(t_6 \cdot s_2)$

Figure 3: Result and provenance for Q_{region}

2 PROVENANCE MODELS

A key challenge in evaluating provenance systems is that they often support different provenance models [12, 15, 25, 33]. These models differ in at which granularity they track provenance (e.g., tuples or attribute values) and in whether they only track data dependencies (e.g., which input tuple does a query result tuple depend on) or also model how input tuples are combined to produce a result. Furthermore, most models track all alternative derivations of a result while some only track one sufficient one. These differences have important implications for the functionality, performance, and storage requirements of provenance systems. This section introduces the provenance models considered in this work. Our goal is not to provide a comprehensive survey, but rather to highlight the key trade-off of these models. We refer interested reader to existing surveys for a detailed treatment of provenance models [15, 25, 33, 36].

2.1 Witnesses

The notion of a witness for a query result $t \in Q(D)$ was first introduced by Buneman et al. [12] to denote a subset $w \subseteq D$ of the input database D that is *sufficient* for producing t :

$$t \in Q(w)$$

As already observed in [12], a witness may contain redundant tuples that are not needed to produce the result. To avoid such redundancy, we can require witnesses to be *minimal*:

$$\nexists w' \subset w : t \in Q(w')$$

If the goal of provenance is to merely reproduce query results, then using a single witness as provenance is sufficient.¹

EXAMPLE 2.1. Consider query Q_{region} that returns regions which have departments with employees that earn more than \$100k:

```
SELECT DISTINCT region
FROM Dept d, Emp e
WHERE salary > 100,000;
```

¹In fact, in this case we can also use a witness for the complete query result as in [35] where $w \subseteq D$ is a witness if $Q(w) = Q(D)$. We will call such witnesses “holistic” to distinguish them from witnesses for individual query results. While the distinction to witnesses for individual result tuples seems minor and, in fact, the union of witnesses for all result tuples of a query is a holistic witness, determining minimal witnesses for full query results is unfortunately NP-complete in data complexity [35], even for some subclasses of conjunctive queries.

On the example database from Fig. 1 this query returns two results as shown in Fig. 3. Consider the first result tuple r_1 and the witness shown on the right of the tuple. $\{t_1, s_1\}$ is sufficient for producing r_1 .

2.2 Lineage

When it is important to identify *all* inputs that contributed to a result as part of some derivation, a single witness is insufficient; a more expressive provenance model is needed. *Lineage*² captures the set of all input tuples that participate in at least one derivation of a result. This concept was first coined in [65] and later formalized based on sufficiency and necessity [22].³ For instance, for the query from Ex. 2.1, the lineage of row r_1 is $\{t_1, t_2, s_1\}$ as there are two witnesses (alternative ways to derive r_1): $\{t_1, s_1\}$ and $\{t_2, s_1\}$.

2.3 K-relations and Provenance Polynomials

Lineage identifies which tuples contributed to a result, but does not explain *how* they contributed. Provenance polynomials address this limitation by explicitly encoding the derivation structure of query results. Green et al. [28] introduced K-relations, a generalization of classical relations where tuples are annotated with elements from a commutative semiring K . A semiring consists of a set K and two binary operations (addition and multiplication) over K that have to fulfill a set of algebraic laws: (i) both operations are commutative and associative, (ii) multiplication distributes over addition, and (iii) K contains neutral elements 0_K and 1_K for addition and multiplication, respectively. For an appropriate choice of semiring, K-relations can model set semantics, bag semantics, and several provenance models including Lineage and MWhy-provenance [12] (the set of all minimal witnesses for a query). Green et al. [28] further demonstrated that the semiring $\mathbb{N}[X]$ of *provenance polynomials*, whose elements are polynomials over variables X representing input tuples, subsumes all provenance models expressible in the K-relational model. Using the provenance polynomial semiring $\mathbb{N}[X]$, each input tuple is annotated with a distinct variable. These annotations are propagated during query evaluation such that the annotation of a result tuple t encodes both which input tuples contributed to it and how these tuples were combined. Here, addition represents alternative derivation (e.g., projection or union), while multiplication represents joint use of tuples (e.g., join).

EXAMPLE 2.2 (PROVENANCE POLYNOMIALS). *Consider again query Q_{region} and its result shown in Fig. 3. The provenance polynomial for tuple $r_1 = (\text{North})$ encodes two alternative derivations: joining t_1 with s_1 or joining t_2 with s_2 . Furthermore, the result was subjected to duplicate elimination (**DISTINCT**, represented as δ in the polynomial).*⁴

Provenance polynomials provide a strong guarantee: given the polynomial associated with a query result, we can determine how the result would change if the multiplicities of input tuple were modified. Importantly, the polynomials remain valid even when

input tuples are deleted or their multiplicity is changed.⁵ This makes provenance polynomials particularly useful for view maintenance and reverse data management [44], where the goal is to determine input modifications that achieve a desired change to the output.

EXAMPLE 2.3 (MAINTAINING RESULTS). *Continuing with Ex. 2.2, consider the polynomial for tuple r_1 :*

$$\delta(t_1 \cdot s_1 + t_2 \cdot s_1)$$

Let us assume that we deleted the input tuple t_1 (setting its multiplicity to zero) and increased the multiplicity of the input tuple s_1 to 2 (there are two copies in the input). Will $r_1 = (\text{North})$ still be in the result? To answer this question, we evaluate the polynomial (function δ returns 1 for any non-zero input) by substituting each variable with the multiplicity of the corresponding input tuple:

$$\delta(0 \cdot 2 + 1 \cdot 2) = \delta(0 + 2) = \delta(2) = 1$$

Sine the result is non-zero, tuple r_1 is still in the query result.

2.4 Coarse-grained Over-Approximations

Tracking provenance at the granularity of rows provides detailed information about how query results are derived. However, for some use cases this is overkill, e.g., when it is enough to have a subset of the input database (witness) on which a query produces the same result as on the full database. For such applications, provenance can be captured at a coarser granularity, trading precision for smaller provenance size. For instance, Niu et al. [52] propose *provenance sketches*, which track provenance at the granularity of fragments induced by a horizontal partitioning of a relation. This is also related to summarization of provenance polynomials [3, 4].

EXAMPLE 2.4. *For instance, consider a range-based horizontal partition of table Dept on attribute region. The coarse-grained provenance of Q_{region} using the partition would contain the two fragments $f_{North} = \{s_1, s_2, s_5\}$ and $f_{East} = \{s_4\}$ as only these fragments contain provenance. This is an over-approximation of lineage as the tuple s_5 does not belong to the lineage of neither result tuple r_1 nor r_2 .*

3 USE CASES

The usefulness of a provenance system ultimately depends on the tasks it enables. Different applications require different forms of provenance and place different demands on provenance-capture, storage, and querying mechanisms. This section reviews representative use cases for (relational) data provenance and identify their requirements in terms of provenance models and system functionality. These requirements motivate the workload dimensions of PBench. In particular, they illustrate why evaluating provenance systems solely based on capture performance is insufficient: a provenance representation that is efficient to capture may incur additional overhead for downstream tasks.

3.1 Debugging Query Results

By tracking which outputs of a query depend on which inputs, database provenance is a useful tool for debugging. *Backward debugging* starts from an unexpected result and seeks the input tuples responsible for it. For instance, assume that in our running example

²The term lineage is also used in the probabilistic database literature to refer to a different type of provenance, namely Boolean lineage formulas over input tuples. In the K-relational model, this corresponds to semiring $\text{PosBool}(X)$. To avoid confusion, we do not use the term lineage for $\text{PosBool}(X)$ in this work.

³For some approaches, Lineage is used in a looser sense and may not contain tuples from all possible derivations of a result under some operations, e.g., nested subqueries.

⁴Technically, the δ operator is part of an extension of the semiring framework that models duplicate elimination [6].

⁵Handling the insertion of new input tuples requires more expressive provenance models extending the K-relational framework. [30, 31, 39, 67]

we consider result tuple r_3 suspicious, because its salary value is unreasonably low. The provenance of this tuple unveils that this is due to the low salary of t_3 . *Forward debugging* starts from known problematic data and determines which query results may be affected. For instance, if some data is imported from an unreliable source, then we would like to understand which query results are untrustworthy because they depend on data from this source. In both cases of debugging, provenance narrows the search space by identifying relevant data dependencies.

In terms of provenance models, debugging requires provenance that captures all contributing tuples across all derivations. Single witnesses are insufficient because they do not identify all problematic data dependencies, whereas lineage is sufficient and provenance polynomials provide additional information about how tuples were combined. Note that we assume that the query itself is correct. Fixing issues with the query requires orthogonal techniques which are beyond the scope of this benchmark [5, 10, 11, 14, 23, 41, 42, 46, 63].

3.2 Reproducing Results

Reproducibility of computational experiments, at the bare minimum requires that rerunning an experiment in a different environment should produce the same result. Provenance enables the creation of “*reproducibility packages*” [16, 62] which contain sufficient data and instructions to ensure this. For computations that involve a relational database, a reproducibility package should only include subsets of the database(s) accessed by the experiment that are sufficient for guaranteeing that all executed statements return the same result as in the original execution [54, 55].

Because a witness is by definition sufficient to regenerate a query result, witnesses are the least expressive provenance model required for reproducibility. More expressive models such as lineage and provenance polynomials are also sufficient, but generally capture information that is unnecessary for this task.

EXAMPLE 3.1. Assume that query Q_{region} (Ex. 1.1) is part of a computational experiment. A reproducibility package for this experiment only needs to include a single witness for this query. For example for Q_{region} , such a “holistic” witness can be generated by computing the union of a witness for each result tuple (Fig. 3):

$$w = \{t_1, t_6, s_1, s_4\}$$

3.3 Probabilistic Databases

Probabilistic databases model data uncertainty through a probability distribution over possible worlds. A commonly used model is tuple-independent databases [64], where each tuple is associated with a marginal probability of existence. Evaluating queries over such databases requires computing the marginal probability for each possible result tuple to appear in the query’s output. Provenance plays an essential role in this computation [61, 64]. Provenance polynomials provide a symbolic representation of the derivations of a result tuple and can be used to compute marginal probabilities under bag semantics. Under set semantics, Boolean formulas over variables representing input tuples are sufficient.⁶ Unlike debugging or reproducibility, probabilistic query evaluation requires knowing the exact derivation structure of query results.

⁶These formulas correspond to semiring $PosBool(X)$ in the K-relational framework.

Consequently, witnesses and lineage are insufficient, and provenance polynomials (or Boolean provenance formulas under set semantics) are required.

EXAMPLE 3.2 (PROBABILISTIC DATABASES). Consider the polynomial for result tuple r_1 for query Q_{region} . By substituting addition with logical or and multiplication with logical and, we get the “Lineage” formula:

$$(t_1 \wedge s_1) \vee (t_2 \wedge s_1) = s_1 \wedge (t_1 \vee t_2)$$

Assuming input tuple probabilities $p(t_1) = 0.8$, $p(t_2) = 0.9$, $p(s_1) = 0.5$, the marginal probability of r_1 is the same as the probability of its Lineage formula evaluating to true [64]:

$$p(r_1) = p(s_1) \cdot (1 - ((1 - p(t_1)) \cdot (1 - p(t_2)))) = 0.5 \cdot (1 - 0.2 \cdot 0.1) = 0.49$$

3.4 View Maintenance & View Update

View maintenance and view-update problems require reasoning about how changes to input data affect query results. This dependence on derivation structure makes them natural applications of provenance polynomials. In view maintenance [32], the output of a query should be updated to reflect an update to the database. Similarly, in what-if analysis [8, 48] we want to determine the effect of a hypothetical change to the database. For any deletion of a subset of inputs, the provenance polynomial of a query result can be used to evaluate the impact of this deletion by replacing the variables for deleted tuples with zero and reevaluating the polynomials. Thus, provenance polynomials provide a solution to maintain views for positive relational algebra queries where updates are restricted to deletions.⁷ In view update (*How-to* analysis [45]), a desired change to the output of a view should be translated into an update to the database such that running the view’s query over the updated database yields the requested update to the view’s result. If we limit updates to deletions, then this is referred to as *delete propagation* [13, 37]. Using provenance polynomials, this boils down to constraint-solving [44, 45]: find an assignment of input variables that determines which tuples to delete such that the result tuple polynomials evaluate to a desired multiplicity. Lineage and, thus, also a single witness, is insufficient for view maintenance and update, as it does not track how input tuples were combined, necessary for determining how input tuple deletions affect results.

EXAMPLE 3.3 (WHAT-IF AND HOW-TO). Suppose Alice does not work for the company any more (tuple t_1 is deleted). To determine the impact of this update on Q_{region} , we can replace t_1 with 0 in the polynomial for every result tuple. Any result tuple whose polynomial now evaluates to zero will no longer be in the result. Consider the polynomial for r_1 :

$$\delta(0 \cdot s_1 + t_2 \cdot s_1) = \delta(t_2 \cdot s_1)$$

Thus, r_1 is still in the result. Now consider a *How-to* query. To delete result tuple r_1 but keep r_2 , we have to solve the following constraints:

$$t_1 \cdot s_1 + t_2 \cdot s_1 = 0 \qquad t_6 \cdot s_2 > 0$$

For example, one possible solution is to delete s_1 (setting $s_1 = 0$) and keep all other tuple multiplicities the same as in the original database.

⁷With the extensions from [6], aggregate queries are also supported. Furthermore, there are provenance models that also deal with negation and can support insertions [30, 40]

3.5 Optimizing Data Management

Beyond explanation, provenance also enables new optimizations in relational databases. Since the provenance of a query is sufficient to reproduce its result, it acts as a compact summary of the data relevant to that query. For example, the provenance of a query Q_1 can be materialized and reused to accelerate the evaluation of a related query Q_2 by executing Q_2 over Q_1 's provenance rather than the full database [52, 66]. Provenance can also guide physical design decisions: data that frequently appears in query provenance can be prioritized for placement in faster storage or caches [26]. These applications typically only require identifying data that may influence a result, not how it contributes to the result. Consequently, lightweight provenance models such as witnesses and coarse-grained approximations (e.g., provenance sketches [52]) are often sufficient and preferable due to their compactness.

EXAMPLE 3.4 (OPTIMIZATION). Assume we have captured coarse-grained provenance for Q_{region} as explained in Ex. 2.4 containing the fragments f_{North} and f_{East} for regions "North" and "East". The following query differs from Q_{region} only in using a stricter WHERE condition. This query can be evaluated on $\{f_{North}, f_{East}\}$ as its provenance is contained in Q_{region} 's provenance.

```
SELECT DISTINCT region
FROM Dept d, Emp e
WHERE salary > 115,000;
```

4 SYSTEM OVERVIEW

For the benchmark evaluation we only consider reasonably complete systems. We selected a representative set of systems regarding the architectural / algorithmic decisions. Table 1 compares these systems GProM [7], ProvSQL [60], SmokedDuck [47] and SQLProv [50] on their system architecture & how provenance is captured, supported provenance models, supported SQL features, and provenance representation and query capabilities. We show how these systems capture provenance for example query Q_{emp} in [43].

4.1 GProM

GProM [7] compiles provenance requests into pure SQL, supporting PostgreSQL, SQLite, DuckDB, MonetDB, and Oracle as backends. For positive relational algebra, GProM computes a flat-relational encoding of Provenance Polynomials while for more general queries its model encodes at least Lineage. In addition, GProM supports coarse-grained provenance. GProM uses a propagation approach: the user selects which attributes of input tables to treat as provenance and GProM instruments the input query Q to construct a SQL query Q_{rew} that propagates these annotations such that each result tuple of Q_{rew} pairs a result tuple of Q with the selected attributes of tuples from its provenance. GProM exposes provenance capture as a new SQL query feature and, thus, full SQL can be used to combine capture and querying of provenance in a single query.

4.2 ProvSQL

ProvSQL [60] is a PostgreSQL extension that adds support for computing provenance polynomials encoded as circuits. In these circuits each node is identified through an automatically generated UUID. To enable provenance tracking for a table, the user has to call a UDF `add_provenance` which generates a UUID for each row in the table as an identifier for the variable representing this row. Under the hood, this adds a new column `ProvSQL` to the table storing

these UUIDs. ProvSQL adds a new background worker process to PostgreSQL that logs circuit gates to memory-mapped files. To capture provenance, queries are instrumented by adding calls to UDFs that create new circuit gates and edges by sending messages to the background worker using IPC. In the result of the instrumented query an additional attribute `provsql` stores the UUID of a gate that is the root of the subcircuit encoding the polynomial for the row. ProvSQL provides UDFs for constructing a string representation of a circuit rooted at a gate and for evaluating the encoded polynomial in a target semiring. For the later the user provides a "provenance mapping" that assigns semiring elements to variables.

4.3 Smoked Duck

SmokedDuck [47] extends the execution engine of DuckDB to enable capture. Lineage tracking in SmokedDuck consists of two phases: (1) during query execution, the instrumented versions of a physical operator in SmokedDuck logs to memory a mapping table that records for each output t of the operator which input tuples (their rowids) of the operator contributed; (2) Once lineage has been captured, the user can request the lineage for a particular result tuple (identified by its rowid) or to get a mapping from output to input rowids for all outputs. For that SmokedDuck joins the mapping tables to trace back from the output to the input tables one operator at a time. The system also supports provenance polynomials.

4.4 SQLProv

SQLProv [50] captures fine-grained provenance for SQL by instrumenting queries. SQLProv is implemented on top of PostgreSQL, but the approach could be ported relatively easily to other databases. The system supports both tuple-level provenance as well as attribute-level provenance. Provenance is captured in two phases. In the first phase, the input query is instrumented to create tuple identifiers and to log mapping tables between input and output tuple identifiers for operators, similar to SmokedDuck. This is done by injecting UDFs into the query that insert tuples into logging tables. In the second phase, the input query is instrumented to operate on sets of provenance tuple (or attribute value) identifiers instead of data values. UDF function calls are added to load mappings from the logging tables and computations on values are replaced with union operations on provenance identifier sets. In the result of this instrumented query each row (attribute value) stores the set of input tuple (attribute value) identifiers in its provenance.

4.5 System Comparison

The evaluated systems differ along three primary dimensions: provenance-capture strategy, provenance representation, and storage architecture. GProM and ProvSQL construct provenance during query execution and expose it immediately to the user, whereas SQLProv and SmokedDuck use a two-phase workflow in which dependency information is first recorded and provenance is reconstructed later. This design can be advantageous when provenance is captured once but queried multiple times, as the capture cost is incurred only once. The systems also differ in how provenance is represented. GProM encodes provenance as a mapping table that associates each result tuple with contributing tuples, potentially duplicating the result tuple if necessary. For each result tuple,

Category	Property	ProvSQL	GProM	SmokedDuck	SQLProv
Architecture	Type Backend	Extension PostgreSQL	Middleware Multiple	Modified Execution Engine DuckDB	Extension PostgreSQL
Capture	Workflow Strategy	Single-phase Circuit propagation	Single-phase Annotation propagation	Two-phase I. Mapping + II. Backtrace	Two-phase I. Mapping + II. Replay
Provenance	Lineage Witnesses Provenance Polynomials Coarse-grained	✓ ✓ ✓ ✗	✓ ✓ ✓(SPJ) ✓	✓ ✓ ✓ ✗	✓ ✓ ✗ ✗
SQL Support	SPJ (+DISTINCT) Aggregation Set Operations Nested Queries Additional Features	✓ Partial Set only ✗ -	✓ ✓ ✓ ✓ Top-K	✓ ✓ ✓ ✓ Top-K	✓ ✓ ✗ ✓ Window, Recursive, Top-K
Storage	Additional Storage	Circuits in mmap'ed files	None	In-memory Maps	Logging Tables

Table 1: Comparison of evaluated provenance systems.

ProvSQL uses an additional attribute to store the UUID of the root gate of a circuit for the row’s provenance polynomial, the circuits themselves are stored in memory-mapped files. SQLProv represents provenance of a result tuple as a set of tuple ids encoded as a Postgres array value. SmokedDuck displays the row ids of provenance tuples. Finally, the systems differ in their storage requirements. GProM captures provenance by running rewritten queries and requires no extra storage, whereas extra storage is needed for ProvSQL (the mmap files storing circuits), SQLProv (the logging tables) and SmokedDuck (in memory mapping tables).

5 PBENCH

We now introduce PBench, our benchmark for evaluating relational provenance systems. PBench evaluates the entire provenance life-cycle, including capture, storage, querying, and downstream use. The benchmark consists of three complementary workload families that stress different aspects of provenance systems: (i) application-oriented tasks derived from common provenance use cases, (ii) workloads that vary provenance size and structural properties, and (iii) workloads that exercise increasingly complex SQL constructs. Together, these workloads expose trade-offs among provenance expressiveness, capture overhead, storage requirements, and downstream usability. We first discuss the benchmark design, evaluation metrics, and the data generation process before presenting the individual benchmark tasks. As in TPC-H and other benchmarks, a scale factor P_{SF} controls the size of the input database.

5.1 Design Considerations

PBench organizes benchmark tasks into three workload families.

Use-case-driven workloads. These workloads evaluate complete provenance workflows derived from practical applications such as debugging, reproducibility, and probabilistic query processing. They measure not only provenance capture but also the cost of consuming provenance in a task-specific manner.

Provenance-centric workloads. These workloads isolate the effect of provenance characteristics such as total provenance size, provenance size per result, intermediate provenance volume relative to final provenance size, and factorization. They reveal how provenance representation affects scalability.

Notation	Description
P_{SF}	Scale factor for the benchmark
$P_{\% Q(D) }$	Fraction of query result tuples for which provenance is requested
$P_{\% D }$	Fraction of input tuples which should be traced to a query’s result
P_{tot}	Total provenance size for all result tuples of a query
P_{res}	Average size of the provenance of a single result tuple
P_{fact}	Ratio between the size of the sum-of-product of provenance polynomials for a query and the size of polynomials factorized based on the structure of the query
$P_{wit/lin}$	Ratio between the size of a single witness and the size of the full lineage of a query
T_{e2e}	End-to-end task runtime (provenance capture + querying + additional task-specific steps)
T_{cap}	Runtime for capturing provenance
T_{query}	Runtime of querying provenance data
T_{task}	Runtime of a task that uses queried provenance
T_{prep}	Runtime for preparing a database to be able to capture provenance
$S_{capture}$	Storage overhead incurred by a system to run a task
S_{db}	Storage overhead for preparing a database for provenance capture

Figure 4: Parameters and metrics of the benchmark

Query-centric workloads. These workloads evaluate how query structure and SQL features influence provenance capture and storage. They stress systems with increasingly complex relational operators and language constructs.

Fig. 4 shows notation we use for parameters (that are varied in individual tasks) and metrics used in PBench.

5.2 Metrics

For each task, PBench reports *execution time* and *storage overhead* $S_{capture}$. Depending on the workload, end-to-end execution time T_{e2e} is decomposed into provenance capture (T_{cap}), provenance querying (T_{query}), and task execution (T_{task}). For systems that separate provenance capture into logging and reconstruction phases (e.g., SmokedDuck and SQLProv), capture time is further decomposed into $T_{cap-log}$ and $T_{cap-trace}$. Storage overhead ($S_{capture}$) includes all additional data generated for provenance tracking, whether written to disk (e.g., ProvSQL stores provenance circuits as external files) or stored as database tables (e.g., SQLProv logs provenance in relational tables). Furthermore, some systems require augmenting the input database to enable provenance tracking. For these systems, we also report the one time storage overhead of augmenting the base data (S_{db}) and the time (T_{prep}) needed to prepare the input database.

Table	#Col.	#Rows	Table	#Col.	#Rows
R_{fp}^n	7	$n \cdot P_{SF}$	R_{grp}^n	4	$n \cdot P_{SF} \cdot 10^3$
R_{join}^c	4	$5 \cdot P_{SF} \cdot 10^4$	$R_{grpsize}^n$	4	$n \cdot P_{SF} \cdot 10^3$
R_{join}^s	4	$25 \cdot P_{SF} \cdot 10^3$	R_{fac}^1, R_{fac}^2	5	$P_{SF} \cdot 10^4$
R_{join}^1, R_{join}^2	5	$P_{SF} \cdot 10^4$	R_{graph}	5	$P_{SF} \cdot 10^5$

Table 2: Synthetic tables used in the benchmark

5.3 Data Generation

PBench combines standard decision-support data with synthetic datasets specifically designed to control provenance characteristics. We use the *TPC-H* dataset scaled by the benchmark scale factor P_{SF} , together with 17 synthetically generated tables that allow us to independently vary provenance size, provenance factorization, query selectivity, and query structure. Table 2 summarizes the characteristics of all synthetic tables used in the benchmark. All attributes in synthetic tables are uniformly distributed. These tables are not intended to model realistic data distributions, but to provide fine-grained control over characteristics such as total provenance size, provenance factorization, and query selectivity. Table R_{fp}^n , for $n \in \{1k, 100k, 2M, 10M\}$, varies the number of distinct attribute values, allowing control over the provenance size per output tuple. Tables R_{join}^c , R_{join}^s , R_{join}^j and R_{join}^{jj} contain attributes with at most 1K distinct values and are used in join conditions to control join selectivity and result cardinality. Tables R_{grp}^n (and $R_{grpsize}^n$) are designed for aggregation queries, where n controls the number of groups and the number of tuples per group, respectively. Tables R_{fac}^1 & R_{fac}^2 are used to control the degree of provenance factorization. Finally, R_{graph} represents an edge relation of a graph with 1K nodes and 10^5 edges, and is used to evaluate recursive queries such as reachability. We show the SQL code for every benchmark query and provide additional details about these queries such as intermediate result sizes in [43].

5.4 Use-case-driven Workloads

Use-case-driven workloads evaluate complete provenance workflows rather than provenance capture alone. Many applications require additional computations over captured provenance, and the cost of this computation depends on how provenance is represented, stored, and queried. In particular, tasks that require full input or output rows may impose additional work on systems that return only tuple identifiers, since these identifiers must be joined back with the base tables or query result to retrieve attribute values.

Unless stated otherwise, each workload follows a common evaluation protocol: (1) capture provenance, (2) query the captured provenance to retrieve the relevant rows when needed, and (3) perform the task-specific computation over the retrieved data. As mentioned above, we decompose the end-to-end execution time as follows:

$$T_{e2e} = T_{cap} + T_{query} + T_{task} \quad (\text{Task execution time breakdown})$$

where T_{cap} is the provenance-capture time, T_{query} is the time to query or retrieve provenance, and T_{task} is the time for task-specific computations. Systems may combine these steps unless explicitly stated otherwise. We describe the system-specific implementations of these workflows in [43].

5.4.1 Backward Debugging. T1: Backward Debugging starts from suspicious query results and retrieves the input tuples responsible for producing them. This workload evaluates whether a system can efficiently compute provenance for a selected subset of query results, rather than the entire output. The workflow follows the standard protocol, where the last step filters provenance to return explicit data for the selected result tuples. Systems may optimize this workflow by pushing the selection into the capture phase. We evaluate backward debugging using query Q_{BDebug} , a join followed by group-by aggregation that produces 1K result tuples. We vary the fraction of erroneous results, $P_{\%|Q(D)|} \in \{1\%, 10\%, 50\%\}$. The number of provenance tuples for $P_{\%|Q(D)|} = 1\%$ is $\sim 10K$ ($1\% \cdot 1K \cdot (1K + 10)$).

5.4.2 Forward Debugging. Forward debugging starts from problematic input tuples and identifies query results that may be affected by them. This workload evaluates the ability of a provenance system to search provenance in the opposite direction: from inputs to outputs. The task, **T2: Forward debug - impacted rows**, returns all result rows whose provenance contains at least one erroneous input tuple. We use two query templates: Q_{FD1} , a join followed by aggregation, and Q_{FD2} , a join query. We vary $P_{\%|D|}$, the fraction of erroneous input tuples, from 1% to 5% in table R_{grp}^{1k} . Additional forward debugging variants are described in [43]: determining which queries in a workload are affected by erroneous inputs and mapping each erroneous input to the result tuples it affects.

5.4.3 Reproducibility. Reproducibility workloads evaluate whether a system can compute a subset of the input data sufficient to reproduce a query result. Unlike debugging tasks, reproducibility requires actual input rows rather than only tuple identifiers, to rerun the query. The workflow therefore consists of (i) capturing and materializing provenance ($T_{cap} + T_{query}$), and (ii) re-executing the query on the materialized provenance (T_{task}). We separate these steps to reflect the creation and subsequent use of a reproducibility package.

We consider two reproducibility tasks that differ in the provenance model used. **T3: Reprod. - lineage** uses full lineage as the reproducibility package. It is evaluated with query template Q_{Lin} , which joins R_{fp}^{10M} with R_{join}^c and applies group-by aggregation with aggregation function `min`. The total size of lineage is $\sim 10M \cdot P_{SF}$ ($(10M + 10K) \cdot P_{SF}$) tuples. **T4: Reprod. - witness** requires extracting a single witness, which is sufficient for reproducibility and can be much smaller than full lineage. Since none of the evaluated systems directly produces a single witness, this task also captures the cost of extracting a witness from a more expressive provenance representation. We use the query template Q_{wit} , a group-by aggregation on top of a join, and vary join selectivity through parameter $n \in \{1k, 100k, 2M\}$ which affects the left input of the join (table R_{fp}^n), resulting in witness sizes of $P_{wit/lin} \in \{0.01\%, 1\%, 20\%\}$ relative to the full lineage, $\sim 10M \cdot P_{SF}$. An additional reproducibility task based on coarse-grained provenance is described in [43].

5.4.4 Provenance Polynomials. Several use cases, including probabilistic query processing and hypothetical reasoning (e.g., what-if and how-to queries), require provenance polynomials because they depend on the derivation structure of query results. While provenance capture provides the foundation for these usecases, they

require additional functionality beyond basic capture and querying, which is only supported by some systems (e.g., FADE [48] built on SmokedDuck for what-if queries, and ProvSQL for probabilistic query evaluation). We therefore focus in this workload on the common prerequisite: capturing provenance polynomials. In **T5: Provenance Polynomials**, systems have to compute provenance polynomials for SPJ queries $Q_{\text{join}[1]}$, $Q_{\text{join}[2]}$, and $Q_{\text{join}[3]}$, which vary the number of joins, and query Q_{joinagg} , which is a group-by aggregation over a join. We measure capture time (T_{cap} with the breakdown into $T_{\text{cap-log}}$ and $T_{\text{cap-trace}}$ for 2-phase systems) and storage overhead (S_{capture}). The size of the polynomial for each result tuple is constant in the data and linear in the number of joins for SPJ queries $Q_{\text{join}[1]}$, $Q_{\text{join}[2]}$ and $Q_{\text{join}[3]}$. For Q_{joinagg} the polynomial of a result tuple contains about respectively, $\sim 20K$ operations (gates).

5.5 Provenance-centric Workloads

These workloads isolate the impact of provenance characteristics on system performance. They evaluate how provenance size and structural properties affect the cost of provenance capture. For these tasks, systems are only required to record sufficient information during query execution to enable subsequent provenance reconstruction unless stated otherwise. Accordingly, for two-phase systems such as SQLProv and SmokedDuck, we report only the runtime of the first (capture) phase, $T_{\text{cap-log}}$.

5.5.1 Provenance Size per Result Tuple. T6: Prov. per Result isolates the effect of provenance size per result tuple while keeping the total provenance size ($P_{\text{tot}} = 10M \cdot P_{\text{SF}}$) fixed. We vary the average provenance size per result (P_{res}) from 10 to 10^6 using grouping aggregation (QFPAgg) and **INSTINCT** (QFPDist) queries. This workload evaluates how provenance representations scale when the same total provenance becomes concentrated in fewer result tuples.

5.5.2 Varying Total Provenance Size. T7: Ttl. Prov. Size evaluates scalability with respect to the total amount of captured provenance. We vary P_{tot} from $100K \cdot P_{\text{SF}}$ to $10M \cdot P_{\text{SF}}$ using two complementary query templates. Query QVPGN fixes the average provenance size per result while varying the number of result tuples, whereas QVPGS fixes the number of result tuples and varies P_{res} , the average provenance size per result.

5.5.3 Varying Intermediate Provenance Size. Many provenance systems generate substantially more provenance during intermediate query evaluation than appears in the final query result. If the final provenance is relatively small, then two phase systems that reconstruct provenance top-down (as SmokedDuck does) have an advantage over bottom-up reconstruction as in SQLProv (or single phase bottom-up capture as in GProM). To evaluate these differences we also measure the cost of the 2nd phase for such systems for this workload. **T8: Interm. Prov. Size** evaluates the cost of this intermediate provenance by varying $P_{\text{surviving}}$, the fraction of the provenance of intermediate query results that ends up in the provenance of the final query result, e.g., $P_{\text{surviving}} = \frac{1}{3}$ means that one third of the intermediate provenance is in the final provenance for the query. We use query Q_{TopK} which is a top-k query over a sorted group-by result. We vary k , the number of retrieved tuples, resulting in the $P_{\text{surviving}} \in \{\frac{1}{2 \cdot P_{\text{SF}}}, \frac{1}{10 \cdot P_{\text{SF}}}, \frac{1}{100 \cdot P_{\text{SF}}}, \frac{1}{1,000 \cdot P_{\text{SF}}}\}$.

5.5.4 Provenance Polynomial Factorization. T9: Factorization evaluates a system's ability to exploit factorization in provenance polynomials. We vary the factorization ratio,

$$P_{\text{fact}} = \frac{\#p}{\#SOP(p)},$$

where $\#p$ is the size of a factorized provenance polynomial and $\#SOP(p)$ is the size of its equivalent sum-of-products (SOP) representation, measured by the number of variable occurrences. Smaller values therefore indicate greater compression through factorization. Note that for factorized provenance, we consider DAGs (circuits) rather than expression trees, i.e., identical sub-expressions do not have to be repeated. We compare query pairs that produce equivalent provenance with different factorization. Query Q_{JD} produces a flat SOP polynomial, whereas Q_{DJ} pushes duplicate elimination below the join to produce a factorized polynomial of the form $\delta(\sum_i r_i) \cdot \delta(\sum_j s_j)$ where $\sum_i r_i$ is from the left input of the join and $\sum_j s_j$ is from the right input. We consider two variants with different number of distinct values by using different attributes to project on yielding $P_{\text{fact}} = \frac{1}{200 \cdot P_{\text{SF}}}$ and $P_{\text{fact}} = \frac{1}{20 \cdot P_{\text{SF}}}$ for Q_{DJ} . We also consider factorization with aggregation using queries Q_{JA} and Q_{AJ} . Here, Q_{AJ} preaggregates the left input before the join yielding a factorization structure of $\delta(\sum_j \delta(\sum_i r_i) \cdot s_j)$. For Q_{AJ} we have $P_{\text{fact}} \sim \frac{1}{10}$.

5.6 Query-centric Workloads

These workloads evaluate how the structure of SQL queries affect the performance of provenance capture (T_{cap}) and storage overhead (S_{capture}). Unlike the use-case-driven workloads, they focus solely on provenance capture rather than downstream provenance consumption. As in the provenance-centric workloads, systems are only required to record sufficient information to enable later provenance reconstruction.⁸ Therefore, two phase systems report only the runtime of the first phase ($T_{\text{cap-log}}$).

5.6.1 TPC-H. T10: TPC-H measures provenance capture for all 22 queries of the TPC-H benchmark. We include TPC-H because it is a widely used decision-support benchmark containing diverse SQL constructs, including complex joins, aggregations, and nested subqueries. The TPC-H scale factor is set to the benchmark scale factor P_{SF} .

5.6.2 Subqueries. T11: Sub-Q evaluates provenance capture for queries containing **WHERE** subqueries. Different systems adopt different strategies for handling subqueries. For example, GProM rewrites them into lateral joins, whereas SQLProv does not capture provenance for table appearing exclusively inside **WHERE** subqueries. We use query template Q_{WSub} , group-by-aggregation with **EXISTS** in **WHERE** clause(see [43]) and vary the number of subqueries from one to three. Systems produce unfactorized provenance polynomials, $\sum_i r_i$, for each result tuple without provenance for tables in subqueries, as subqueries work as filter conditions. However, provenance capture for tables including those appearing in subqueries produces factorized polynomials in the format of $\sum_i (r_i \cdot$

⁸Just recording what queries are executed is not considered sufficient. That is, the information captured here should record actual provenance relationships at the data level, but it is acceptable if an additional post-processing step is needed to merge the recorded information to produce the full provenance of a query.

$(\sum_j s_j) \cdot (\sum_k t_k) \cdot (\sum_l u_l)$ for each output tuple of Q_{WSub} with three subqueries. To enable a fair comparison, we evaluate GProM both with its native behavior, where the provenance of all query tables are captured, and with provenance capture restricted to relations outside subqueries. Additional query-centric workloads covering language features such as argmin, window functions, and recursive queries are described in [43].

6 EXPERIMENTAL EVALUATION

We use PBench to compare four representative relational provenance systems: SmokedDuck [47], ProvSQL [60], SQLProv [50], and GProM [7]. Among these, ProvSQL, SQLProv, and GProM run on PostgreSQL, while GProM and SmokedDuck support DuckDB. The evaluated systems differ substantially in their provenance models, capture strategies, and provenance representations, allowing us to study the trade-offs exposed by PBench. For systems that split capture into two phases, namely SQLProv and SmokedDuck, we report the cost of the first phase ($T_{\text{cap-log}}$) and, where applicable, the cost of reconstructing provenance in the second phase ($T_{\text{cap-trace}}$). In the figure, blue percentages above bars indicate the fraction of total capture time spent in the first phase. For tasks that require returning full provenance rows, systems that represent provenance as row ids must additionally join these identifiers with the base tables or query results to retrieve attribute values; GProM can avoid this step by directly producing provenance as relational data. All experiments were run on a machine with an Apple M2 Max CPU with 12 cores and 64GB of RAM, running macOS with Darwin Kernel 23.6.0. We use PostgreSQL 16.3 and DuckDB 1.0.0⁹

6.1 Preparing the Database For Capture

We first evaluate the one-time cost of preparing the input database for provenance capture. This step is required only by ProvSQL and SQLProv. ProvSQL augments each input table with provenance tokens that map tuples to variables used in provenance polynomials. SQLProv creates provenance-specific versions of each table whose values are identifiers propagated during the second phase of provenance computation.

For the full PBench database at $P_{\text{SF}} = 1$, the original database size is roughly 3.2GB. ProvSQL requires about 122 seconds to add provenance-token columns to all relations in the database, increasing storage by approximately ~ 2.2 GB. SQLProv requires about 105 seconds to build its provenance-specific tables, increasing the database size to approximately 13.3GB. On *TPC-H* with scale factor 1, ProvSQL requires about 37 seconds and adds 0.4GB of storage, whereas SQLProv requires about 39 seconds and increases storage from 2.27GB to 7.33GB. Thus, systems that require modifying or duplicating the input database introduce substantial one-time preparation and storage overhead before query-level provenance capture begins.

6.2 Use-case-driven Workloads

We first evaluate workloads that require provenance to be consumed by downstream tasks. These experiments highlight that provenance-capture cost alone does not determine end-to-end performance: representation choices and the ability to push task-specific constraints into provenance capture can substantially affect runtime.

T1: Backward Debugging. Fig. 5a reports end-to-end runtime for backward debugging, where provenance is retrieved for a fraction $P_{\%|Q(D)|}$ of query result tuples. GProM is the only evaluated system that can both restrict provenance capture to selected output tuples and directly return provenance as relational rows¹⁰. As a result, its runtime scales with $P_{\%|Q(D)|}$. In contrast, the other systems first capture provenance for all result tuples and then retrieve the relevant input rows using row identifiers, making their runtime largely insensitive to the fraction of suspicious outputs. The breakdown in Fig. 5b shows that capture time dominates these workflows. While SmokedDuck has the lowest first-phase capture time, the cost of reconstructing provenance in the second phase outweighs this advantage for end-to-end backward debugging.

T2: Forward Debugging. Fig. 5c and Fig. 11b show the cost of identifying result tuples affected by erroneous input rows ($P_{\%|D|} = 5\%$) for aggregation and join queries respectively. SmokedDuck is omitted for this experiment, see Sec. B.1.4 for a detailed explanation. For the aggregation query (Fig. 5c), SQLProv has lower capture cost than ProvSQL, but higher end-to-end runtime because affected rows must be identified by scanning provenance arrays for erroneous tuple identifiers. This illustrates a key PBench finding: compact or efficient capture is not sufficient if the resulting representation is expensive to query for downstream tasks.

Pengyuan deleted:

Boris says: @Pengyuan: explain

because Added

T3: Reproducibility and T4: Reproducibility. Fig. 5d and 5e compare reproducibility using lineage and using a single witness. Since none of the evaluated systems directly computes a single witness, we simulate the potential benefit by prefiltering the input table to approximate the effect of witness-based capture. This gives an optimistic estimate, as a real system would still need to identify the witness over the full input database. Nonetheless, this experiment demonstrates that witness-based reproducibility can provide significant runtime and storage savings when a query has many alternative derivations. In Fig. 5e, the witness is only 0.01% of the full lineage, demonstrating the potential benefit of provenance models tailored to reproducibility.

T5: Provenance Polynomials. Fig. 5f shows the runtime of capturing provenance polynomials for the systems that support this model. For GProM, we request a string encoding of polynomials using `WITH SEMIRING COMBINER NX` to make the comparison closer to systems that explicitly materialize polynomial representations. SmokedDuck has the lowest first-phase capture time, but once reconstruction and polynomial construction are included, GProM achieves the best end-to-end runtime. ProvSQL benefits from a compact

⁹We use DuckDB 1.0.0 because this is the version on which SmokedDuck is based.

¹⁰Some versions of SmokedDuck do support backtracking provenance in the second phase of capture for individual result rows, but there are limitations in the current implementation, see [43].

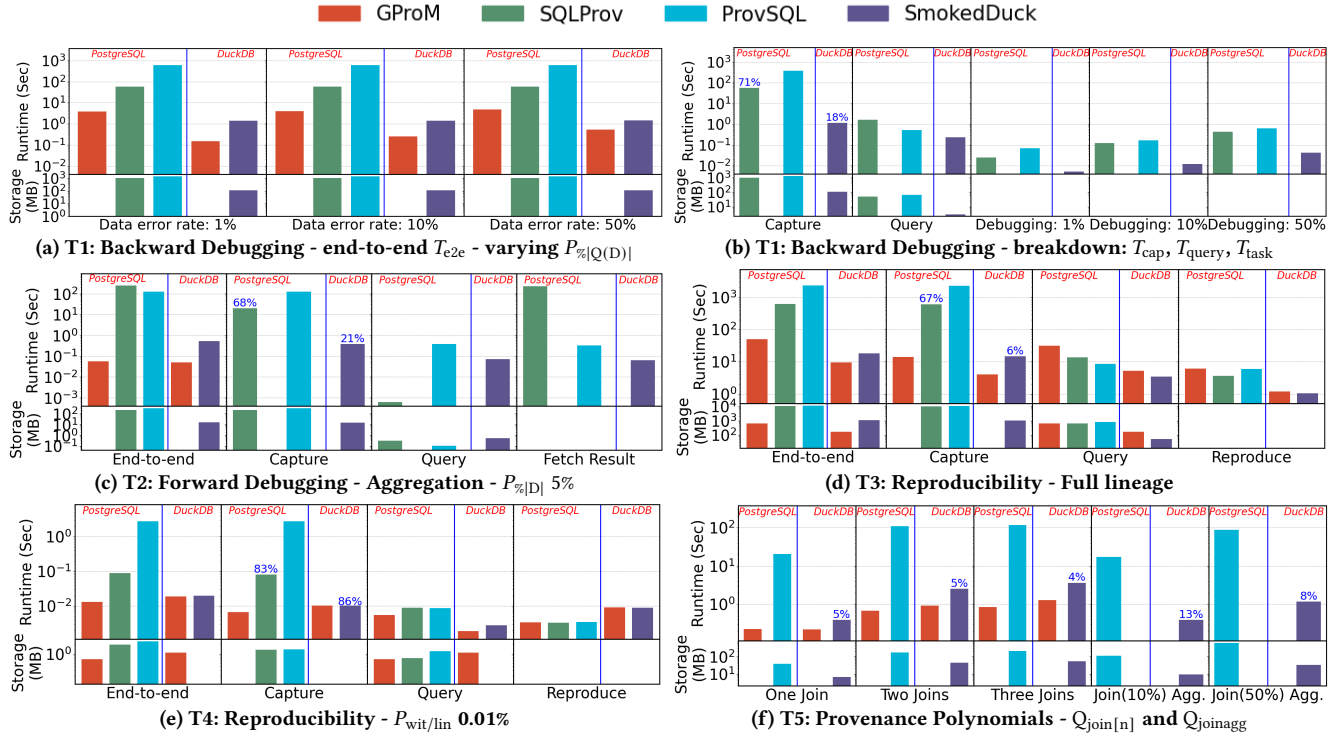


Figure 5: Experimental Evaluation for Use-case Specific Tasks

circuit encoding, but its capture cost is dominated by the overhead of creating circuit gates through interprocess communication and writing them to memory-mapped files.

Discussion. The use-case workloads show that end-to-end provenance performance depends on both capture strategy and representation. SmokedDuck is highly efficient when only first-phase capture is considered, making it attractive when provenance must be collected eagerly for many queries. However, tasks that require reconstructed provenance can shift the advantage toward single-phase systems such as GProM. Moreover, systems that expose provenance only as tuple identifiers incur additional costs when applications require full rows. These results demonstrate why provenance systems should be evaluated using complete workflows rather than capture time alone.

6.3 Provenance-centric Workloads

In this section, we evaluate systems using the benchmark tasks that vary provenance structure and size. As these tasks focus exclusively on provenance capture, we report only the first capture phase ($T_{cap-log}$) for two-phase systems such as SQLProv and SmokedDuck, unless stated otherwise. As a reference, we also report the runtime of each query without provenance capture.

T6: Provenance Size per Result Tuple. Fig. 6b reports runtime and storage overhead while varying the average provenance size per result tuple (P_{res}) and keeping the total provenance size (P_{tot}) fixed. We evaluate both the aggregation query Q_{FPagg} and the **DISTINCT** query Q_{FPdist} . Since ProvSQL’s circuit-construction cost is

dominated by the total number of provenance operations, its runtime is largely insensitive to how provenance is distributed across result tuples. For Q_{FPagg} , SmokedDuck is the fastest system. For both SmokedDuck and GProM, reducing P_{res} increases the absolute runtime, but the relative runtime overhead over the query without provenance (the dashed line) decreases. For Q_{FPdist} , GProM outperforms SmokedDuck due to employing a specific optimization for **DISTINCT** queries: when **DISTINCT** is the last operator, it can sometimes be removed for provenance tracking.

T7: Varying Total Provenance Size. Fig. 6c shows the runtime when varying the total provenance size P_{tot} by varying the total number of groups in an aggregation query. All systems scale roughly linearly with total provenance size, indicating that total provenance volume is a primary driver of capture cost. Varying group size yields similar trends (see [43]).

T8: Varying Intermediate Provenance Size. Fig. 6d evaluates the impact of intermediate provenance by varying $P_{surviving}$, the ratio between intermediate and final provenance size, using queries with a **LIMIT** clause. We vary the number of tuples returned by the query and report the full capture runtime ($T_{cap} = T_{cap-log} + T_{cap-trace}$). The percentages above the bars indicate the fraction of the total runtime spent in the first phase.

As the **LIMIT** threshold increases, SmokedDuck’s total runtime also increases, while its first-phase capture cost remains nearly constant. This behavior reflects its two-phase design: the first phase records provenance mappings independent of the query output size, whereas the second phase reconstructs provenance only for the returned tuples. Consequently, SmokedDuck benefits when the

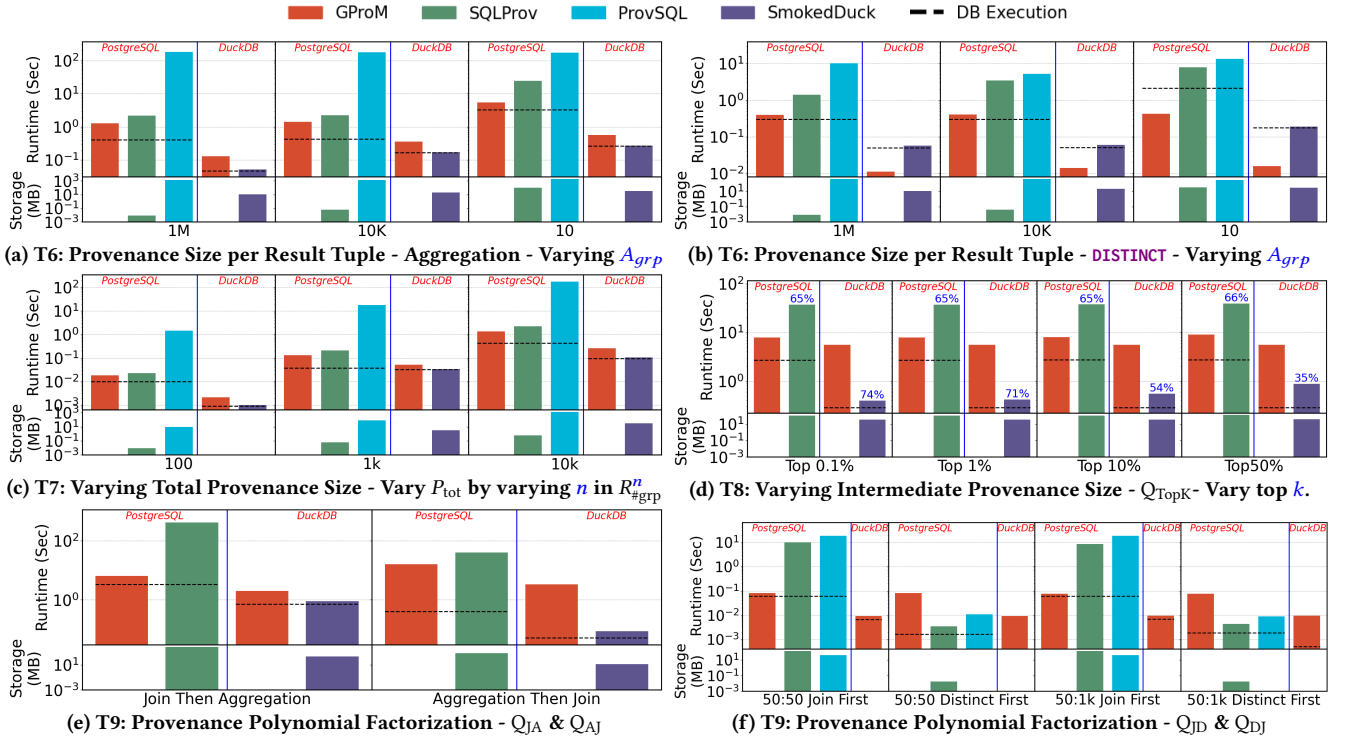


Figure 6: Experimental evaluation for Provenance Structure & Size Tasks

final provenance is much smaller than the intermediate provenance. In contrast, GProM and SQLProv maintain nearly constant runtime across all settings, as they exploit a bottom-up provenance tracing strategy which computes provenance for all tuples regardless of the number of tuples returned by a query, making their runtime insensitive to the **LIMIT** threshold.

Pengyuan deleted:

Pengyuan deleted:

Boris says: Actually only GProM shows really much of an increase which has to be explained

because Previous plots shows only the phase I. Now the new plot show total phases presenting the increase when more data pass the filter condition.

Pengyuan deleted:

Boris says: We are missing the phase I percentages in the plots!

because Replaced with the new plots show the total time plus phase I percentage. Previous plot only shows the phase I runtime.

because need to be deleted, since now we use the top-k queries for the intermediate. And the topk in query-specific task of benchmark description and exp will be removed.

Pengyuan deleted: The results for the subtask varying the number of joins is shown in [43]. because Deleted, this is removed from benchmark description. we no longer need this

T9: Provenance Polynomial Factorization. In Fig. 6e, we change the factorization structure of an aggregation over a join (Q_{JA}) by pushing the aggregation into the join (Q_{AJ}), thus, factorizing the provenance. Recall that P_{fact} the size of factorized versus non-factorized provenance. For Q_{AJ} , the P_{fact} is $\sim 1/2$ and SQLProv and SmokedDuck benefit significantly from factorization, up to $\sim 10X$

faster, because compared with Q_{JA} , there is less data to be materialized in the first phase. GProM performs the worst, since it cannot exploit the factorization and Q_{AJ} introduces another level of aggregation computation which make it more expensive to compute than Q_{JA} .

Pengyuan deleted: As the factorization is not very extreme ($P_{fact} \approx 1/2$ the original provenance size) even systems that theoretically benefit from factorization do not show significant runtime improvements. ProvSQL is excluded from this experiment as it does not support joining on the result of an aggregation with another table. because Updated above

Pengyuan deleted:

Boris says: That we should fix to make the factorization more extreme

because Run an new query, now we can see the changes.

Fig. 6f show the result for duplicate elimination after a join (Q_{JD}) versus pushing the duplicate elimination operator into both sides of the join (Q_{DJ}). SmokedDuck is excluded as the provenance it produces for **DISTINCT** is incorrect. This leads to an extreme factorization of $P_{fact} \approx \frac{1}{10k}$. As GProM cannot exploit factorization, its runtime is roughly the same for both queries while for the factorized version SQLProv and ProvSQL which benefit from factorization outperform GProM (a three OOM reduction in runtime).

Discussion. Discussion. These experiments show that the choice

of capture strategy has a major impact on performance. When only sufficient information needs to be recorded for later provenance reconstruction, two-phase systems have a clear advantage because they execute only the first capture phase. In this setting, SmokedDuck consistently achieves the lowest capture overhead across

most workloads. We also find that exploiting provenance factorization can substantially reduce runtime and storage requirements, but these benefits become significant only when the provenance is highly factorized. For moderate levels of factorization, the higher per-tuple capture overhead of SQLProv and ProvSQL outweighs the gains from more compact provenance representations.

6.4 Query-centric Workloads

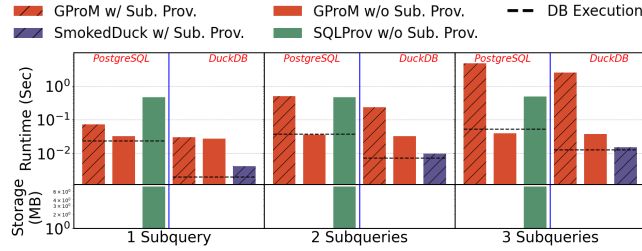


Figure 7: T11: Subqueries - Vary n in Q_{WSub}

We finally evaluate workloads that stress query structure and SQL language features. As in the provenance-centric workloads, we report only $T_{cap-log}$ for two-phases system such as SmokedDuck and SQLProv, unless noted otherwise.

Query	Database: PostgreSQL					Database: DuckDB				
	GProM	SQLProv	ProvSQL	SmokedDuck	DB Execution	GProM	SQLProv	ProvSQL	SmokedDuck	DB Execution
	T_{cap}	$T_{cap-log}$	$T_{cap-trace}$	$S_{capture}$		T_{cap}	$T_{cap-log}$	$T_{cap-trace}$	$S_{capture}$	
Q ₁	2.896	28.208	—	327.6	445.8	2661	0.909	0.160	0.432	22.65
Q ₂	0.271	0.121	0.005	0.039	—	—	0.183	0.014	0.360	7.876
Q ₃	0.473	0.825	0.203	2.604	—	—	0.177	0.062	0.093	19.90
Q ₄	2.201	0.579	0.161	2.863	—	—	0.351	0.061	0.342	18.74
Q ₅	0.272	0.401	0.096	0.511	—	—	0.177	0.071	0.124	5.410
Q ₆	0.307	1.297	1.669	6.332	1.030	12.24	0.071	0.031	0.002	0.524
Q ₇	0.973	0.641	0.075	0.425	0.280	0.543	0.335	0.152	0.791	72.48
Q ₈	0.199	—	—	—	—	—	0.485	0.043	0.083	3.972
Q ₉	0.591	3.077	3.399	21.90	4.296	47.44	2.408	0.153	0.255	15.51
Q ₁₀	0.801	1.624	0.734	9.441	—	—	0.388	0.120	0.104	16.74
Q ₁₁	1.649	0.369	0.016	4.039	—	—	6.272	0.007	0.030	1.062
Q ₁₂	0.428	—	—	—	1.112	5.143	0.124	0.061	0.051	4.711
Q ₁₃	1.601	11.50	68.62	97.97	—	—	0.642	0.169	0.539	41.12
Q ₁₄	0.313	—	—	—	—	—	0.064	0.034	0.026	7.157
Q ₁₅	1.750	1.291	0.149	1.262	—	—	—	0.058	0.070	5.593
Q ₁₆	—	1.078	0.369	8.504	—	—	—	0.022	0.050	3.361
Q ₁₇	3.062	4.200	0.007	10.39	—	—	0.512	0.054	0.119	0.578
Q ₁₈	—	2.153	0.004	0.046	—	—	—	0.364	1.467	154.2
Q ₁₉	0.037	0.051	0.002	0.019	0.025	0.000	0.140	0.071	0.075	14.61
Q ₂₀	2.375	—	—	—	—	—	0.238	0.055	0.032	4.222
Q ₂₁	284.7	0.925	0.047	0.318	—	—	—	0.174	0.457	45.99
Q ₂₂	26.32	0.101	0.014	0.372	—	—	—	0.031	0.009	3.863

Figure 8: T10: TPC-H: Runtime (T_{cap} in seconds) and Storage overhead ($S_{capture}$ in MB). $T_{cap-log}$ ($T_{cap-trace}$) indicates the runtime for phase I (II).

T10: TPC-H. Fig. 8 shows provenance capture runtime for TPC-H queries. For several queries, provenance size is moderate, making SQLProv and ProvSQL more competitive with the exception of Q_1 and Q_{13} , where provenance size is large (e.g., ProvSQL writes over ~ 2.5 GB of data to disk for query Q_1). SmokedDuck is up to ~ 7 times faster than GProM and 2-3 times faster for most queries. Furthermore, for queries with (multiple) nested subqueries where each subquery has relatively large provenance, the capability of a system to factorize provenance is critical: in fact, for some of these queries SmokedDuck outperforms GProM by several OOM.

Pengyuan deleted:

Boris says: BASED ON NEW RESULTS: Furthermore, for queries with (multiple) nested subqueries where each subquery has relatively large provenance, the capability of a system to factorize provenance is critical: in fact, for some of these queries SmokedDuck outperforms GProM by several OOM.

because Added this paragraph to paper.

T11: Subqueries. Fig. 7 shows the result for Q_{WSub} as the number of subqueries in the WHERE clause increases. For GProM and SmokedDuck, capturing provenance for tables inside subqueries causes runtime to grow rapidly, because each additional subquery increases the total provenance by roughly an order of magnitude. When provenance is not captured for tables appearing only inside subqueries, all evaluated systems remain comparatively stable, since the subqueries primarily act as filters and the computation is less expensive.

Discussion. The query-centric workloads show that query structure can be as important as provenance size. For complex queries with relatively small provenance, ProvSQL and SQLProv are closer to GProM, typically within a small constant factor rather than orders of magnitude slower. However, nested subqueries expose important differences in representation and rewriting strategy. GProM suffers from rewriting nested subqueries into LATERAL joins, which are not always optimized well by PostgreSQL, and from using a flat, non-factorized representation that enumerates all combinations of tuples drawn from the provenance of each subquery. These results reinforce the need for query-centric workloads in PBench: provenance-system performance depends not only on how much provenance is captured, but also on how query structure shapes that provenance.

Pengyuan deleted:

Boris says: show

because Added the subquery exp.

7 RELATED WORK

Data provenance [15, 25, 33] traces the origin of data that contributes to the result of a query.

Provenance Models. Several provenance models have been proposed to provide a formal semantics for explaining query results under different classes of queries [15, 25, 33]. A witness [12] is a subset of database that is sufficient to generate the same result. Lineage [18–20, 22], capturing all inputs that contribute to a result of query, was first introduced in [65] and then was formalized based on sufficiency and necessity [22]. Why-provenance [12, 22] identifies which sets of input tuples (witnesses) are sufficient to generate the result. Where-provenance [12] traces the input tuples from which the result is derived. Green et al. [28, 29] introduced K-relations and demonstrated that the semiring $\mathbb{N}[X]$ whose elements are polynomials over variables X that represent tuples generalizes all other provenance models expressible as semirings (including, e.g., lineage and why-provenance). Niu et al [52] proposes provenance sketches, a coarse-grained provenance model. The approach captures sketches over-approximating the provenance of a query and utilizes these sketches to speed-up subsequent queries. [3, 4] discuss how to summarize provenance polynomials by merging multiple variables into one new variable. Why-not [14] methods

explain missing results using provenance. Detailed introductions to relational provenance models can be found in [15, 25, 33, 36].

Provenance Use Cases. Important use case of provenance include the creation of "*reproducibility packages*" for computational experiments [16, 62] that include sufficient input data to guarantee that computations (queries in the relational case) produce the same result as in the original experiment [54, 55]. Provenance serves a tool for debugging. [51] present a system for debugging transactions with provenance. Provenance also plays an essential role in probabilistic databases [58, 60, 61, 64], where the polynomial formulas provide a symbolic representation of tuple derivations and are used to compute the marginal probability of query result tuples. In view maintenance under delete propagate scenarios [13, 37], provenance polynomials are sufficient for maintaining results as exploited, e.g., in FADE [48]. Provenance polynomials have also been used in Howto analysis [31, 44, 45, 67] for hypothetical reasoning.

Relational Provenance Systems. Provenance can be captured by annotating data and propagating these annotations using relational queries or by extending the database system [38, 53]. Systems like GProM [7], Perm [27], Smoke [57], SmokedDuck [47], Links [24], ProvSQL[59], SQLProv [50], Trio [2], DataProv [56], and DBNotes[9] capture provenance for SQL queries.

8 CONCLUSIONS AND DISCUSSION

We present *PBench*, a comprehensive benchmark for relational provenance systems specifically designed to stress-test such systems, compare existing systems on a leveled playing field, and empower developers of such system to better understand the strengths and weaknesses of their systems. We use *PBench* to evaluate several SOTA relational provenance systems. The results of this evaluation demonstrate that (i) paying attention to optimizing the per tuple cost of provenance capture is important as systems like ProvSQL and SQLProv use efficient data structures, but are outperformed by several OOM by competitors that use more lightweight capture mechanisms on provenance-heavy queries; (ii) as expected, extending the execution engine of a database instead of relying on extensibility mechanisms as done by SmokedDuck results in the most performance capture implementation, but if taking into account the need to reconstruct provenance through joins in phase II of capture can outweighs this benefits for provenance-heavy queries; (iii) the flat "1NF" representation use by GProM is effective for many queries as it does not require any UDTs to encode provenance, but degrades performance significantly for queries whose provenance permits concise factorizations, e.g., joining results of duplicate elimination or aggregation and queries with multiple nested subqueries; (iv) systems that can incorporate querying of provenance into provenance capture have advantages for use cases where only a fraction of the provenance, e.g., provenance for some rows of a query result, should be returned; (v) perhaps more attention should be paid to in which format provenance is exposed to the user as many use cases require access to the actual rows in the provenance instead of just sets of row identifiers; (vi) as simpler provenance models are sufficient for some tasks, significant performance gains are possible by directly capturing provenance in these simpler models instead of generating expressive provenance

(e.g., polynomials) and then extracting the simpler provenance from these results.

REFERENCES

- [1] 2007. 2nd Provenance Challenge. <https://openprovenance.org/provenance-challenge/SecondProvenanceChallenge.html>.
- [2] Parag Agrawal, Omar Benjelloun, Anish Das Sarma, Chris Hayworth, Shubha U. Nabar, Tomoe Sugihara, and Jennifer Widom. 2006. Trio: A System for Data, Uncertainty, and Lineage. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006*, Umeshwar Dayal, Kyu-Young Whang, David B. Lomet, Gustavo Alonso, Guy M. Lohman, Martin L. Kersten, Sang Kyun Cha, and Young-Kuk Kim (Eds.). 1151–1154.
- [3] Eleanor Ainy, Pierre Bourhis, Susan B. Davidson, Daniel Deutch, and Tova Milo. 2015. Approximated Summarization of Data Provenance. In *CIKM*, James Bailey, Alistair Moffat, Charu C. Aggarwal, Maarten de Rijke, Ravi Kumar, Vanessa Murdock, Timos K. Sellis, and Jeffrey Xu Yu (Eds.). 483–492.
- [4] Eleanor Ainy, Pierre Bourhis, Susan B. Davidson, Daniel Deutch, and Tova Milo. 2016. PROX: Approximated Summarization of Data Provenance. In *EDBT, Evaggelia Pitoura, Sofian Maabout, Georgia Koutrika, Amélie Marian, Letizia Tanca, Ioana Manolescu, and Kostas Stefanidis (Eds.)*. 620–623.
- [5] Shatha Algarni, Boris Glavic, Seokki Lee, and Adriane Chapman. 2025. Efficient Query Repair for Aggregate Constraints. *Proceedings of the VLDB Endowment* (2025).
- [6] Yael Amsterdamer, Daniel Deutch, and Val Tannen. 2011. Provenance for Aggregate Queries. In *PODS*. 153–164.
- [7] Bahareh Arab, Su Feng, Boris Glavic, Seokki Lee, Xing Niu, and Qitian Zeng. 2018. GProM - A Swiss Army Knife for Your Provenance Needs. *IEEE Data Eng. Bull.* 41, 1 (2018), 51–62.
- [8] Sepehr Assadi, Sanjeev Khanna, Yang Li, and Val Tannen. 2016. Algorithms for Provisioning Queries and Analytics. In *ICDT*, Wim Martens and Thomas Zeume (Eds.), Vol. 48. 18:1–18:18.
- [9] Deepavali Bhagwat, Laura Chiticariu, Wang Chiew Tan, and Gaurav Vijayvargiya. 2005. An annotation management system for relational databases. *VLDBJ* 14, 4 (2005), 373–396.
- [10] Nicole Bidot, Melanie Herschel, and Aikaterini Tzompanaki. 2015. Efficient Computation of Polynomial Explanations of Why-Not Questions. In *CIKM*, James Bailey, Alistair Moffat, Charu C. Aggarwal, Maarten de Rijke, Ravi Kumar, Vanessa Murdock, Timos K. Sellis, and Jeffrey Xu Yu (Eds.). 713–722.
- [11] Nicole Bidot, Melanie Herschel, Katerina Tzompanaki, et al. 2014. Query-Based Why-Not Provenance with NedExplains. In *EDBT*.
- [12] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330.
- [13] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2002. On Propagation of Deletions and Annotations through Views. In *PODS*. 150–158.
- [14] Adriane Chapman and H. V. Jagadish. 2009. Why Not?. In *SIGMOD*. 523–534.
- [15] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474.
- [16] Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. 2016. Reprozip: Computational reproducibility with ease. In *Proceedings of the 2016 international conference on management of data*. 2085–2088.
- [17] Gao Cong, Wenfei Fan, Floris Geerts, Jianzhong Li, and Jizhou Luo. 2012. On the Complexity of Annotation Propagation and View Update Analyses. *IEEE TKDE* 24, 3 (2012), 506–519.
- [18] Yingwei Cui and Jennifer Widom. 2000. Lineage Tracing in a Data Warehousing System. In *ICDE*. 683–684.
- [19] Yingwei Cui and Jennifer Widom. 2000. Practical Lineage Tracing in Data Warehouses. In *ICDE*. 367–378.
- [20] Yingwei Cui and Jennifer Widom. 2001. Lineage Tracing for General Data Warehouse Transformations. In *VLDB*. 471–480.
- [21] Yingwei Cui and Jennifer Widom. 2001. *Run-time translation of view tuple deletions using data lineage*. Technical Report. Stanford.
- [22] Yingwei Cui, Jennifer Widom, and Janet L. Wiener. 2000. Tracing the Lineage of View Data in a Warehousing Environment. *TODS* 25, 2 (2000), 179–227.
- [23] Ralf Diestelkämper, Seokki Lee, Melanie Herschel, and Boris Glavic. 2021. To not miss the forest for the trees - A holistic approach for explaining missing answers over nested data. In *SIGMOD*. 405–417.
- [24] Stefan Fehrenbach and James Cheney. 2018. Language-integrated provenance. *Sci. Comput. Program.* 155 (2018), 103–145.
- [25] Boris Glavic. 2021. Data Provenance - Origins, Applications, Algorithms, and Models. *Foundations and Trends® in Databases* 9, 3-4 (2021), 209–441.
- [26] Boris Glavic, Pengyuan Li, Ziyu Liu, Dieter Gawlick, Vasudha Krishnaswamy, Danica Porobic, and Zhen Hua Liu. 2024. Towards an Objective Metric for Data Value Through Relevance. In *Proceedings of the 14th Conference on Innovative Data Systems*.
- [27] Boris Glavic, Renée J. Miller, and Gustavo Alonso. 2013. Using SQL for Efficient Generation and Querying of Provenance Information. In *In Search of Elegance in the Theory and Practice of Computation - Essays Dedicated to Peter Buneman, Val Tannen, Limsoon Wong, Leonid Libkin, Wenfei Fan, Wang-Chiew Tan, and Michael P. Fourman (Eds.)*, Vol. 8000. 291–320.
- [28] Todd J. Green, Gregory Karvounarakis, and Val Tannen. 2007. Provenance Semirings. In *PODS*. 31–40.
- [29] Todd J Green and Val Tannen. 2017. The Semiring Framework for Database Provenance. In *PODS*. 93–99.
- [30] Erich Grädel and Val Tannen. 2017. Semiring Provenance for First-Order Model Checking. *arXiv preprint arXiv:1712.01980* (2017).
- [31] Erich Grädel and Val Tannen. 2020. Provenance analysis for logic and games. *Moscow Journal of Combinatorics and Number Theory* 9, 3 (2020), 203–228.
- [32] Ashish Gupta, Inderpal Singh Mumick, and Venkatramanan Siva Subrahmanian. 1993. Maintaining views incrementally. In *SIGMOD Record*, Vol. 22. 157–166.
- [33] Melanie Herschel, Ralf Diestelkämper, and Houssem Ben Lahmar. 2017. A survey on provenance: What for? What form? What from? *VLDBJ* (2017), 1–26.
- [34] Rinke Hoekstra and Paul Groth. 2014. PROV-O-Viz-understanding the role of activities in provenance. In *International Provenance and Annotation Workshop*. Springer, 215–220.
- [35] Xiao Hu and Stavros Sintos. 2024. Finding Smallest Witnesses for Conjunctive Queries. In *ICDT*.
- [36] G. Karvounarakis and T.J. Green. 2012. Semiring-Annotated Data: Queries and Provenance. *SIGMOD Record* 41, 3 (2012), 5–14.
- [37] Benny Kimelfeld, Jan Vondrák, and Ryan Williams. 2012. Maximizing Conjunctive Views in Deletion Propagation. *TODS* 37, 4 (2012), 24:1–24:37.
- [38] Sven Köhler, Bertram Ludäscher, and Yannis Smaragdakis. 2012. Declarative datalog debugging for mere mortals. In *International Datalog 2.0 Workshop*. Springer, 111–122.
- [39] Seokki Lee, Sven Köhler, Bertram Ludäscher, and Boris Glavic. 2016. Implementing Unified Why- and Why-Not Provenance Through Games. In *TaPP*.
- [40] Seokki Lee, Bertram Ludäscher, and Boris Glavic. 2018. PUG: a framework and practical implementation for why and why-not provenance. *VLDBJ* 28, 1 (2018), 47–71.
- [41] Jinyang Li, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. Query Refinement for Diversity Constraint Satisfaction. *PVLDB* 17, 2 (2023), 106–118.
- [42] Jinyang Li, Alon Silberstein, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. ERICA: Query Refinement for Diversity Constraint Satisfaction. *PVLDB* 16, 12 (2023), 4070–4073.
- [43] Pengyuan Li, Davood Rafiei, and Boris Glavic. 2027. (supplementary material). https://github.com/pli26/PBench_2027/blob/main/main.pdf.
- [44] Alexandra Meliou, Wolfgang Gatterbauer, and Dan Suciu. 2011. Reverse data management. *PVLDB* 4, 12 (2011).
- [45] A. Meliou and D. Suciu. 2012. Tiresias: The database oracle for how-to queries. In *SIGMOD*. 337–348.
- [46] Chaitanya Mishra and Nick Koudas. 2009. Interactive query refinement. In *EDBT*. 862–873.
- [47] Haneen Mohammed, Charlie Summers, Sugosh V. Kaushik, and Eugene Wu. 2023. SmokedDuck Demonstration: SQLStepper. In *SIGMOD*, Sudipto Das, Ippokratris Pandis, K. Selçuk Candan, and Sihem Amer-Yahia (Eds.). 183–186.
- [48] Haneen Mohammed, Eugene Wu, Alexander Yao, Charlie Summers, Lampros Flokas, Gromit Yeuk-Yin Chan, Subrata Mitra, and Hongbin Zhong. 2024. Fade: More Than a Million What-Ifs Per Second. *PVLDB* 18, 4 (2024), 943–955.
- [49] L. Moreau, B. Ludascher, I. Altintas, R.S. Barga, S. Bowers, S. Callahan, G. Chin Jr, B. Clifford, S. Cohen, S. Cohen-Boulakia, et al. 2007. The First Provenance Challenge. *Concurrency and Computation: Practice and Experience* (2007).
- [50] Tobias Müller, Benjamin Dietrich, and Torsten Grust. 2018. You say ‘what’, i hear ‘where’ and ‘why’: (mis-) interpreting SQL to derive fine-grained provenance. *PVLDB* 11, 11 (2018), 1536–1549.
- [51] Xing Niu, Boris Glavic, Seokki Lee, Bahareh Arab, Dieter Gawlick, Zhen Hua Liu, Vasudha Krishnaswamy, Su Feng, and Xun Zou. 2017. Debugging Transactions and Tracking their Provenance with Reenactment. *PVLDB* 10, 12 (2017), 1857–1860.
- [52] Xing Niu, Boris Glavic, Ziyu Liu, Pengyuan Li, Dieter Gawlick, Vasudha Krishnaswamy, Zhen Hua Liu, and Danica Porobic. 2021. Provenance-based Data Skipping. *PVLDB* 15, 3 (2021), 451–464.
- [53] Xing Niu, Raghav Kapoor, Boris Glavic, Dieter Gawlick, Zhen Hua Liu, and Venkatesh Radhakrishnan. 2017. Provenance-Aware Query Optimization. In *ICDE*. 473–484.
- [54] Quan Pham, Tanu Malik, Boris Glavic, and Ian Foster. 2015. LDV: Light-weight Database Virtualization. In *ICDE*. 1179–1190.
- [55] Quan Pham, Richard Whaling, Boris Glavic, and Tanu Malik. 2015. Sharing and Reproducing Database Applications. *PVLDB* 8, 12 (2015), 1988–1999.
- [56] Paulo Pintor, Rogério Luís de C. Costa, and José Manuel Moreira. 2026. A Dbms-Independent Approach for Capturing Provenance Polynomials Through Query Rewriting. *IEEE Access* 14 (2026), 66114–66133. <https://doi.org/10.1109/ACCESS.2026.3687631>
- [57] Fotis Psallidas and Eugene Wu. 2018. Smoke: Fine-grained Lineage at Interactive Speed. *PVLDB* 11, 6 (2018), 719–732.
- [58] Aryak Sen, Silviu Maniu, and Pierre Senellart. 2026. ProvSQL: A General System for Keeping Track of the Provenance and Probability of Data. In *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE.

- [59] Pierre Senellart, Louis Jachiet, Silviu Maniu, and Yann Ramusat. 2018. Provenance and Probability Management in PostgreSQL. *PVLDB* 11, 12 (2018), 2034–2037.
- [60] Pierre Senellart, Louis Jachiet, Silviu Maniu, and Yann Ramusat. 2018. Provenance and probability management in postgresQL. *PVLDB* 11, 12 (2018), 2034–2037.
- [61] Dan Suciu. 2020. Probabilistic databases for all. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 19–31.
- [62] Dai Hai Ton That, Gabriel Fils, Zhihao Yuan, and Tanu Malik. 2017. Sciunits: Reusable research objects. *arXiv preprint arXiv:1707.05731* (2017).
- [63] Quoc Trung Tran and Chee-Yong Chan. 2010. How to ConQueR why-not questions. In *SIGMOD*. 15–26.
- [64] Guy Van den Broeck, Dan Suciu, et al. 2017. Query processing on probabilistic data: A survey. *Foundations and Trends® in Databases* 7, 3-4 (2017), 197–341.
- [65] Allison Woodruff and Michael Stonebraker. 1997. Supporting Fine-grained Data Lineage in a Database Visualization Environment. In *ICDE*. 91–102.
- [66] Eugene Wu, Fotis Psallidas, Zhengjie Miao, Haoci Zhang, and Laura Rettig. 2017. Combining Design and Performance in a Data Visualization Management System. In *CIDR*.
- [67] Jane Xu, Waley Zhang, Abdussalam Alawini, and Val Tannen. 2018. Provenance Analysis for Missing Answers and Integrity Repairs. *Data Engineering* (2018), 39.

A QUERY TEMPLATES

In this section, we present all query templates used in the benchmark tasks.

Boris says: @Pengyuan please describe parameters and give a brief explanation for each query including selectivity of join, size of intermediate results where relevant,

A.1 Use-case Specific Tasks

A.1.1 Backward Debugging.

Q_{BDebug} is used in T1: Backward Debugging, shown below:

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rgrp1k JOIN Rjoinc ON (ga = c1to10)
GROUP BY ga
```

Q_{BDebug} joins each tuple from R_{grp}^{1k} with 10 tuples from R_{join}^c and producing a total of 1K result tuples. The total provenance size is $\sim 1M \cdot P_{SF}$ ($1P_{SF} \cdot (M + 10K)$). The debug tracing of each erroneous result tuple produces $\sim 10K \cdot P_{SF}$ tuples.

A.1.2 Forward Debugging.

Q_{FD1} is used in T2: Forward Debugging, shown below:

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rgrp1k, Rjoinc
WHERE ga = c1to10 and ga >= 1 and ga <= 333
GROUP BY ga
```

Q_{FD1} joins each tuple in R_{grp}^{1k} with 10 tuples in R_{join}^c and produces the result of size 334 with 3.34M intermediate tuples generated by join operator. The total size of provenance is $\sim 0.33M \cdot P_{SF}$.

Q_{FD2} is used in T2: Forward Debugging and T13: Forward Debugging, shown below:

```
SELECT ga, va, vb
FROM Rgrp1k, Rjoinc
WHERE ga = c1to10
AND ga >= 667
AND ga <= 1000
```

Both the final result and intermediate provenance sizes of Q_{FD2} are 3.34M. The total size of provenance is the same as Q_{FD1} .

Both join operators in query templates have a 100% selectivity and each tuple from R_{grp}^{1k} has 10 join partners in table R_{join}^c .

A.1.3 Reproducibility.

Q_{Lin} is used in T3: Reproducibility, shown below:

```
SELECT gb, min(va) AS minva
FROM Rfp10M JOIN Rjoinc ON gb = c1to10
GROUP BY gb
```

Q_{Wit} is used in T4: Reproducibility, shown below:

```
SELECT gb, min(va) AS minva
FROM Rfpn JOIN Rjoinc ON gb = c1to10
GROUP BY gb
```

Pengyuan deleted:

Q_{AP} is used in T14: Reproducibility, shown below:

```
SELECT gb, avg(va) AS ava
FROM Rfp10M JOIN Rjoinc ON gb = c1to10
GROUP BY gb HAVING ava < 15
because Described in benchmark appendix
```

Query templates Q_{Lin} accesses R_{fp}^{10M} and R_{join}^c , and outputs 1K result tuples. The join selectivity is 100% and each tuple from R_{fp}^{10M} can join 10 tuples from R_{join}^c . The intermediate tuple size is 100M and the total provenance size is $\sim 10M$.

Q_{Wit} uses R_{fp}^n , reduced size of R_{fp}^{10M} but total number of groups kept the same, varying $n \in \{1K, 100K, 2M\}$ which is the total size of the table. The result sizes are 1K even if we vary the n , and the total provenance sizes are in $\{11K, 101K, \sim 2M\}$. The intermediate result sizes are in $\{110K, \sim 1M, \sim 10M\}$.

A.1.4 Provenance Polynomials.

The three SPJ queries, $Q_{join[1]}$, $Q_{join[2]}$, $Q_{join[3]}$, varying the number of join operations, and one SPJA query, $Q_{joinagg}$ varying the join selectivity, are used for T5: Provenance Polynomials, show as below respectively:

```
SELECT ga
FROM Rgrp1k JOIN Rjoins ON ga = p10
```

```
SELECT ga
FROM Rgrp1k JOIN Rjoins ON ga = p10 JOIN Rjoinj ON ga = c5
```

```
SELECT ga
FROM Rgrp1k JOIN Rjoins ON ga = p10
JOIN Rjoinj ON ga = c5
JOIN Rjoinjj ON ga = cc1
```

```
SELECT ga, avg(va) AS ava
FROM Rgrp1k JOIN Rjoins ON ga = Asel
GROUP BY ga
```

For SPJ queries, $Q_{join[1]}$, $Q_{join[2]}$ and $Q_{join[3]}$, we vary the number of join operators to increase the intermediate size of tuples which are in $\{1M, 5M, 5M\}$, leading to the result sizes are 1M, 5M and 5M. The total size of provenance tuples is $\sim 0.1M$ for all queries. For $Q_{join[2]}$ and $Q_{join[3]}$, even the sizes of total provenance and intermediate result and the result are the same, the number of polynomials of each result tuple are different: 3 and 4 respectively. One purpose to increase the number of join operators is to examine systems' behavior on different polynomial formations as fixing all other settings.

$Q_{joinagg}$ accesses tables R_{grp}^{1K} and R_{join}^s varying the join selectivity in $\{10\%, 50\%\}$, controlled by joining different attributes, A_{sel} in R_{join}^s ,

Task	Query	Tables	Parameter	Join Format	Tot. Prov.	Inter. Res.	Res.
T1: Backward Debugging	QBDDebug	$R_{\text{grp}}^{1k}, R_{\text{join}}^c$	–	$1000\%(1-10)$	~ 1M	10M	1K
T2: Forward Debugging	QFD1	$R_{\text{grp}}^{1k}, R_{\text{join}}^c$	–	$1000\%(1-10)$	~ 0.33M	10M	333
	QFD2	$R_{\text{grp}}^{1k}, R_{\text{join}}^c$	–	$1000\%(1-10)$	~ 0.33M	10M	~ 3.3M
T3: Reproducibility	QLin	$R_{\text{fp}}^{10M}, R_{\text{join}}^c$	–	$1000\%(1-10)$	~ 10M	100M	1K
T4: Reproducibility	QWit	$R_{\text{grp}}^n, R_{\text{join}}^c$	$n = 1K$ $n = 100K$ $n = 2M$	$1000\%(1-10)$ $1000\%(1-10)$ $1000\%(1-10)$	~ 11K ~ 101K ~ 2M	10K 1M 20M	1K 1K 1K
T5: Provenance Polynomials	Qjoin[1]	$R_{\text{grp}}^{1k}, R_{\text{join}}^c$	–	$100\%(1[10\%] - 10)$	~ 0.1M	1M	1M
	Qjoin[2]	$R_{\text{grp}}^{1k}, R_{\text{join}}^c, R_{\text{join}}^j$	–	$500\%(1[10\%] - 10 - 5)$	~ 0.1M	5M	5M
	Qjoin[3]	$R_{\text{grp}}^{1k}, R_{\text{join}}^c, R_{\text{join}}^j, R_{\text{join}}^{jj}$	–	$500\%(1[10\%] - 10 - 5 - 1)$	~ 0.1M	5M	5M
T6: Provenance Size per Result Tuple	QJoinagg	$R_{\text{grp}}^{1k}, R_{\text{join}}^c$	$A_{\text{set}} = p10$ $A_{\text{set}} = p50$	$100\%(1[10\%] - 10)$ $500\%(1[50\%] - 10)$	~ 0.1M ~ 0.5M	1M 5M	100 500
	QFPagg, QFPDist	R_{fp}^{10M}	$ A_{\text{grp}} \in \{10, 1K, 1M\}$	–	10M	–	{10, 1K, 1M}
T7: Varying Total Provenance Size	QVPGN	R_{grp}^n	$n \in \{100, 1K, 10K\}$	–	{0.1M, 1M, 10M}	–	{100, 1K, 1M}
	QVPGS	R_{grpsize}^n	$n \in \{100, 1K, 10K\}$	–	{0.1M, 1M, 10M}	–	1K
T8: Varying Intermediate Provenance Size	QTopK	R_{fp}^{10M}	$k \in \{1K, 10K, 100K, 500K\}$	–	~ 10M	1M	k
T11: Subqueries	QWSub	$R_{\text{grp}}^{100}, R_{\text{join}}^c$	$n \in \{1, 2, 4\}$	–	~ 100K	{1 M, 10 M, 100 M}	100

Table 3: Query templates ($P_{\text{SF}} = 1$) discussed in Sec. A: tables accessed, varied parameters, join format, total provenance size (Tot. Prov.), intermediate result size (Inter. Res.) and result size (Res.)

leading to the intermediate result of size 1M and 5M and result sizes: 100 and 500. The sizes of total provenance are ~ 0.1M and ~ 0.5M.

Pengyuan deleted:

Pengyuan says: Probably we need to vary the parameter for Q_{Joinagg} , adding varying the selectivity? or something else like this.

because Already added

A.2 Provenance Structure & Size

A.2.1 Provenance Size per Result Tuple.

Q_{FPagg} and Q_{FPdist} are used for T6: Provenance Size per Result Tuple, shown as below respectively:

```
SELECT Agrp, avg(va) AS ava, avg(vb) AS avb
FROM Rfp10M
GROUP BY Agrp
```

```
SELECT DISTINCT Agrp
FROM Rfp10M
```

We use Q_{FPagg} and Q_{FPdist} to access a single table R_{fp}^{10M} and control the number of groups (or distinct values) to $P_{\text{SF}} \cdot 10(1K \text{ or } 1M)$ by using the grouping (or distinct) attribute P_{Agrp} to achieve varying provenance size per result tuple.

A.2.2 Varying Total Provenance Size.

Q_{VPGN} and Q_{VPGS} , show as below respectively, are used in T7: Varying Total Provenance Size to vary the total provenance size. Q_{VPGN} fixes the number of tuples per group, 1K, and vary the number of groups $n \in \{100, 1K, 10K\}$, and Q_{VPGS} varies the size of per group $n \in \{100, 1K, 10K\}$, keeping the number of groups fixed to 1K.

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rngrp
GROUP BY ga
```

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rngrpsize
GROUP BY ga
```

The total provenance sizes for both tables ranges in $\{100k \cdot P_{\text{SF}}, 1M \cdot P_{\text{SF}}, 10M \cdot P_{\text{SF}}\}$.

A.2.3 Intermediate Provenance Size.

Q_{TopK}

Pengyuan deleted: and $Q_{\text{Join}\#1}$, $Q_{\text{Join}\#2}$ and $Q_{\text{Join}\#3}$ because Deleted: remove this from the main paper

(shown below) is used for T8: Varying Intermediate Provenance Size.

```
SELECT gb, min(va) AS mva, min(vb) AS mvb
FROM Rfp10M
GROUP BY gc
ORDER BY mva
LIMIT < k
```

Pengyuan deleted:

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rngrp, Rnjoin
WHERE ga = p10
GROUP BY ga

SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rngrp, Rnjoin, Rnjoinj
WHERE ga = p10 AND ga = c5
GROUP BY ga

SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rngrp, Rnjoin, Rnjoinj
WHERE ga = p10 AND ga = c5 AND ga = cc2
GROUP BY ga
```

because Deleted this experiment

For Q_{TopK} , the total size of provenance is $\sim 10M \cdot P_{\text{SF}}$. We vary the number of tuples returned by **LIMIT** operator by controlling the paramter $k \in \{1K, 10K, 100K, 500K\}$ to control the $P_{\text{surviving}}$ in $\{2 \cdot P_{\text{SF}}, 10 \cdot P_{\text{SF}}, 100 \cdot P_{\text{SF}}, 1K \cdot P_{\text{SF}}\}$

A.2.4 Provenance Factorization. Q_{DJ}

```
SELECT t1.da
FROM dj1 t1 JOIN dj2 t2
ON t1.da = t2.da;
```

Q_{JD} used in T9: Provenance Polynomial Factorization is a **DISTINCT** on top of a join. As mentioned in Sec. 5.5.4, we control P_{fact} (how factorizable the provenance is) by select the result attribute **A** and the attribute from **S** we join on (**B**).

δ	Join Condition	$\#P(\sum_i r_i) \cdot (\sum_j s_j)$	$\#SOP(P)(\sum r_i \times s_i)$	P_{fact}
da	R.da = S.da	$2 \cdot 10K \cdot P_{\text{SF}}$	$2 \cdot 10K \cdot P_{\text{SF}} \cdot 10K \cdot P_{\text{SF}}$	$\frac{1}{10K \cdot P_{\text{SF}}}$
	R.da = S.db	$(10K + 200) \cdot P_{\text{SF}}$	$2 \cdot 10K \cdot P_{\text{SF}} \cdot 200 \cdot P_{\text{SF}}$	$\sim \frac{1}{400 \cdot P_{\text{SF}}}$
	R.da = S.dc	$(10K + 10) \cdot P_{\text{SF}}$	$2 \cdot 10K \cdot P_{\text{SF}} \cdot 10 \cdot P_{\text{SF}}$	$\sim \frac{1}{20 \cdot P_{\text{SF}}}$
db	R.db = S.db	$2 \cdot 200 \cdot P_{\text{SF}}$	$2 \cdot 200 \cdot P_{\text{SF}} \cdot 200 \cdot P_{\text{SF}}$	$\frac{1}{200 \cdot P_{\text{SF}}}$
	R.db = S.dc	$(200 + 10) \cdot P_{\text{SF}}$	$2 \cdot 100 \cdot P_{\text{SF}} \cdot 10 \cdot P_{\text{SF}}$	$\sim \frac{1}{20 \cdot P_{\text{SF}}}$
dc	R.dc = S.dc	$(10 + 10) \cdot P_{\text{SF}}$	$2 \cdot 10 \cdot P_{\text{SF}} \cdot 10 \cdot P_{\text{SF}}$	$\frac{1}{10 \cdot P_{\text{SF}}}$

Figure 9: **DISTINCT** P_{fact}

```
SELECT DISTINCT R.A
FROM dj1 AS R JOIN dj2 AS S ON (R.A = S.B)
```

Query Q_{DJ} additionally eliminated duplicates before joining R and S :

```
SELECT DISTINCT R.A
from (SELECT DISTINCT A FROM dj1) AS R
JOIN
(SELECT DISTINCT A FROM dj2) AS S
on R.A = S.B;
```

Note that the two joined tables have the same schema and data distribution, simulating a self-join. Each table has $10K \cdot P_{\text{SF}}$ rows. Both A and B are chosen from one of three attributes: da (one distinct value), db (50 distinct values) and dc ($1k$ distinct values). We evaluate each pair of these three attributes from two tables. For first query Q_{JD} which yields a polynomial of the form $\sum r_i \times s_i$, P_{fact} is 1 independent of the choice of A and B as the provenance for every result tuple is already expressed as a flat sum-of-products. Fig. 9 show P_{fact} for the second query.

Additionally, in this task we use Q_{JA} , an aggregation on top of a join.

Pengyuan deleted:

Pengyuan says: Yes, added, we vary the selectivity as test for task of intermediate result, we test the same three selectivities for different structures Join-agg-having and agg-having-join

Boris says: Are we varying anything in there? If yes, add parameters

because Now, we use the new queries for factorization, varying parameters the groups inside the inner aggregation

```
SELECT Agrp, avg(va) AS ava, avg(vb) AS avb
FROM R10Mfp AS R JOIN Rcjoin AS S ON Agrp = c1to10
GROUP BY Agrp
```

The factorized version, Q_{AJ} , pushes the aggregation into the left side of the join, yielding factorized provenance:

```
SELECT Agrp, sum(sva) / sum(cntva) as ava,
sum(svb) / sum(cntvb) as avb
FROM (
  SELECT Agrp, sum(va) AS sva, sum(vb) AS svb,
  count(va) as cntva, count(vb) as cntvb
  FROM R10Mfp GROUP BY Agrp
) AS R JOIN Rcjoin AS S ON Agrp = c1to10
GROUP BY Agrp
```

For these two query template, we vary the parameter number of groups in the output of group-by $n \in \{10, 1K\}$ controlled by varying the group by attribute A_{grp} . Query Q_{JA} produces a flat sum-of-products $\delta(\sum r_i \cdot s_i)$ and Q_{AJ} returns factorized polynomials of the form $\delta(\sum ((\sum_i r_i) \cdot s_i))$. The number of groups is $1K \cdot P_{\text{SF}}$ and each group has $100K \cdot P_{\text{SF}}$ tuples. For the first aggregation query we have $P_{\text{fact}} = 1$. For the factorized query, $P_{\text{fact}} \sim \frac{1}{2}$.

A.3 Query Constructs & Structure

A.3.1 Subquery. We use the following query template Q_{WSub} varying the number of subqueries in the **WHERE**. The template access two tables: R_{grp}^{100} and R_{join}^c . We fix the result size and vary the total provenance size ($P_{\text{tot}} \in \{10^6, 10^7, 10^8\}$) by increasing 10X for one additional subquery introduced, varying the average provenance size per result tuple ($P_{\text{res}} \in \{10^4, 10^5, 10^6\}$)

```
SELECT ga, avg(va)
FROM R100grp v
WHERE
  EXISTS (SELECT 1 FROM Rcjoin j1 WHERE j1.c1to10 = v.ga)
[
  AND EXISTS (SELECT 1 from Rcjoin j2 where j2.c1to10 = v.ga)
  AND EXISTS (SELECT 1 from Rcjoin j3 where j3.c1to10 = v.ga)
]
GROUP BY ga
```

Table 3 presents the query templates discussed in this section.

B ADDITIONAL DETAILS ON EVALUATED SYSTEMS

We now present additional details on evaluated systems including discussing how these systems capture provenance for an example query.

B.1 Provenance System Setup and Evaluation Protocol

Pengyuan deleted:

Boris says: THIS SECTION NEEDS TO BE MERGED INTO THE OTHER SYSTEM APPENDIX

because Moved here

Pengyuan deleted:

Boris says: I brought comments back, some of this needs to be merged into the systems appendix and then should be deleted

because Done

Boris deleted:For GProM, we can directly work on top of result of capturing since the capture result contains query result and its all contributed data. For ProvSQL, SQLProv and SmokedDuck, when querying the lineage data, we need to map each result id (UUID for ProvSQL and tuid for SQLProv) to all lineage tuples' real ids and to fetch the lineage tuples. Then reproduce the result using the lineage data only.because merge into experiments

Boris deleted:For debugging use case, the capture and query steps are the same as reproducibility. Then comes the debugging phase. For backward debugging, instead of reproducibility, it requires to fetch the specific data that cause the error results. And for forward debugging, we need to check the error result tuples or affected query caused by the error data.because merge into debug or delete

We now discuss in detail what versions of the systems we used in the benchmark and how tasks that go beyond mere capture are implemented in each system.

Pengyuan deleted:

Boris says: Merge into appendix B, it is better to have one appendix covering all systems aspects

because moved Here

B.1.1 GProM. GProM is available on github: <https://github.com/IITDBGroup/gprom>. For PBench, we use git commit 06a1ea4b. GProM allows the user to control what information to propagate as provenance. The default is to propagate all attributes of input tables. For tasks where the goal is to just capture which input

row identifiers contribute, we did only propagate row identifier attributes (e.g., single attribute keys or system row identifiers).

Boris says: @Pengyuan: check what I wrote here

For tasks of forward and backward debugging, we add additional filter operations on top of rewrite queries to fetch the desired tuples. For example, since GProM captures the provenance alongside with the result, backward debugging tasks can be done by filtering for erroneous outputs to fetch the corresponding provenance. For reproducibility tasks, we propagate all attributes of provenance tuples, and retrieve and materialize the provenance data, follow by reproducing queries result using provenance data.

Pengyuan deleted:

Boris says: Discuss how we handle backward + forward and other tasks that go beyond capture

because ADDED

B.1.2 ProvSQL. ProvSQL is available on github: <https://github.com/PierreSenellart/provsql>. The system is constructed as a Postgres extension. We build the extension based on git commit c4a0afd5.

Pengyuan deleted:

Boris says: version X

because Added

Setting up requires adding provenance identifiers to the target tables using `SELECT add_provenance('table')`. During provenance capture, the size of mmap files, storing provenance information for tables, grow rapidly. Once the files reach approximately $\sim 200\text{GB}$, the system is no longer able to handle queries reliably. To ensure a consistent experimental setup, we create a backup of the mmap files after constructing provenance tokens for all required tables. Before each experiment, the current mmap files are discarded and restored from the backup. This guarantees that every query is executed from the same initial provenance state and prevents the continuously growing mmap files from affecting subsequent execution.

Pengyuan deleted:

Boris says: Describe how we cleanup mmaped files

because Added

Pengyuan deleted:

Boris says: Describe how the database setup for ProvSQL, (addprovenance)

because Added

Pengyuan deleted:

Boris says: Explain how we handle tasks that need input or output rows

because Added

For tasks of provenance structure & size and query constructs & structure, we directly execute the benchmark queries and measure the performance. For use-case specific tasks, during provenance capture, we capture provenance using `src_which()` function, which represents provenance as arrays of inputs' identifiers rather than uuid values, which makes it efficient to retrieve the original tuples. For reproducibility tasks, we re-construct query results by re-executing queries over tuples identified from the captured provenance. For backward debugging tasks, after provenance, we query the provenance of the result, and then fetch the provenance tuples of erroneous result. For forward debugging, we query the result data with the corresponding provenance, and then we fetch the impacted

result rows by checking whether erroneous inputs contribute to results.

B.1.3 SQLProv. The full code of SQLProv is not publicly available. We reached out to the authors and were provided with code and instructions that enabled us to run their approach.

Boris says: link to our repository

The SQLProv follows the same execution protocol as ProvSQL, as both systems capture provenance using tuple' identifiers.

B.1.4 SmokedDuck. SmokedDuck is available on github (<https://github.com/cudbg/sd>) And over pipy (<https://pypi.org/project/smokedduck/>). However, the version on pipy crashes with a segfault. We were in contact with the authors and based on their feedback did use the code available in the branch <https://github.com/cudbg/sd/tree/smokedduck-2025-f>

Pengyuan deleted:

Boris says: @Pengyuan: put here

because Added

using git commit 5c0e89d4

Pengyuan deleted:

Boris says: @Pengyuan: add

because Added

The different branches of SmokedDuck differ in how backtracking in phase II operates. Some versions provide an interface where the user provides a single output rowid and the system traces back provenance for that row only using indexes that are build in memory for the captured provenance while others construct a query joins the log tables written during phase I and returns the full provenance. The branch we utilized was using the single rowid backtracking. While we were able to run phase I successfully with this version of SmokedDuck, we encountered issues with phase II. With feedback from the authors we were able to get but this failed for many queries in the benchmark.

SmokedDuck is a forked version of duckdb based on duckdb version 1.0.0. For fairness of comparison we used this version in the evaluation for GProM too.

For reproducibility and forward debugging tasks, SmokedDuck follows the same task protocol as ProvSQL and SQLProv. Similar to these systems, SmokedDuck represents provenance using row identifiers, although it relies on table rowids instead of defined tuple identifiers. For forward debugging, the protocol is mostly the same. However, since SmokedDuck only returns rowids of provenance tuples, in order to complete the tasks, we first associate each result tuple with it provenance using `out_index` field in the retrieved provenance. We then check the affected result rows by identifying whether their provenance contains erroneous input rows.

Boris says: @Pengyuan: explain how we retrieve rows when needed

Pengyuan deleted:

Boris says: @Pengyuan: explain in more detail why forward debugging use case does not work

because added the experiments.

B.2 Example Provenance Capture Process

Fig. 10 shows the capture process for query Q_{emp} from Fig. 1 that we repeat here for convenience:

```
SELECT name, salary
FROM Dept JOIN Emp ON (did = deptid)
WHERE region = 'North';
```

GProM. In Fig. 10, attributes Emp.cid and Dep.did are selected as provenance for input rows. In the result of the instrumented query each tuple is paired with a cid and did value that encodes which input tuples were joined to derive this result. For example, (Alice, 110000) is the result of joining the Emp tuple with cid=1 and department tuple with did=101.

ProvSQL. For the query shown in Fig. 10, the result tuple for Alice is associated with a gate with UUID u_6 which is a +-gate that is the root of the circuit encoding the polynomial $t_1 \cdot s_1$ for this result.

SmokedDuck. In the first phase – shown on the left of SmokedDuck in Fig. 10 – the system executes query and materializes the indices (rowids of a table) of tuples into tables (show in yellow color on the right).

Within the materialized index tables, out_index starts from 0 and increments by 1 for each subsequent tuple. The out_index in the materialized table of an operator serves as the in_index in the materialized table of its parent operator except for table access operator where in_index is the rowid of a tuple. Similarly, lhs_index (rhs_index) in materialized table of a join operator works as in_index.

In the second phase (show on the right), the system joins the materialized index tables to back trace the provenance. It outputs the provenance using rowids of the contributing source tuples. For example, provenance for (Alice, 110000) is {0, 0} (assuming the rowid of two joined tuples are 0 and 0).

Boris says: describe

SQLProv. SQLProv executes the query using the tables built for this phase (in our example Emp1 and Dept1), and logs the tuid (unique identifier for a tuple) of joined tuples into table LogJoin in the first phase. In the second phase, the system bottom-up computes provenance joining original tuples' tuids from tables of Emp2 and Dept2 (built for the second phase) with the materialized tuids and outputs tuids of contributed tuples. For example, the provenance of result tuple (Alice, 110000) is {20, 1}. Note that the tuids for a tuple in constructed tables used by the first and second phases are the same.

C ADDITIONAL BENCHMARK TASKS

C.1 Use-case Specific Tasks

C.1.1 Forward Debugging. In **T12: Forward debug - impacted queries** the system is provided with a set of queries and a set of rows from the database that are considered to be erroneous. This task is then to determine for each query Q in the set whether it has at least one of the erroneous rows in its provenance, i.e., the query's result is potentially affected by the error. We use a query template, Q_{FD1} (shown below),

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R#grp1k, Rjoinc
WHERE ga = c1to10 and ga >= 1 and ga <= 333
GROUP BY ga
```

working on two synthetic tables: $R_{\#grp}^{1k}$ and R_{join}^c . We vary the error data range in table $R_{\#grp}^{1k}$ $P_{\%|D|}$ in 1%, 10% and 30% of total provenance

size $1M \cdot P_{SF}$. The last task **T13: Forward debugging - provenance and result** maps the erroneous provenance tuples with the affected result tuples, and utilizes the same data and query settings as T2.

C.1.2 Reproducibility. **T14: Reeval. - coarse-grained** uses coarse-grained provenance. For this task we consider the scenario of using provenance captured for one query to answer another future query [52, 66]. In this context, coarse-grained provenance has the advantage of small storage size.

Boris deleted: Among all the evaluated systems, GProM supports capturing and utilizing the approximate provenance. To capture provenance in model, GProM encodes fragments, horizontal partitions of relations, as bit-vector – sketches. The system translates the sketches to **WHERE** conditions and apply to queries when using sketches to reproduce the queries result. In this microbenchmark, we examine the performance of GProM implementing reproducibility using approximate provenance. **because Merge into corresponding experiment section**

We use a query Q_{AP} (shown below), and vary the number of fragments, rows of table R_{fp}^{10M} , $10M \cdot P_{SF}$, are grouped into, which determines the granularity of the captured provenance $P_{\#frag} \in \{10, 100, 500\}$.

```
SELECT gb, avg(va) AS ava
FROM Rfp10M JOIN Rjoinc on gb = c1to10
GROUP BY gb HAVING avg(va) < 15
```

As in the reproducibility tasks, the coarse-grained provenance has to be captured and then in a separate step is used to answer a query.

C.2 Query Constructs & Structure Tasks

In this part, we provide additional design of varying structure of queries and what query constructs are used in P Bench.

Pengyuan says: top-k has been move to intermediate provenance size in benchmarks and exp, and the query is described in appendix-query-template. we remove it here

C.2.1 Complex Join Conditions. **T15: Complex Join** uses a join query with a more complex join condition expressed as a disjunctive **WHERE** condition followed by an aggregation.

Pengyuan says: added below

Boris says: What should we present as the purpose of this query?

These complex conditions prevent database engines from exploiting the standard optimizations, such as selection-push-down. Thus, this creates a computationally intensive environment that forces the systems to fully evaluate the join predicates during the capture process. We use a query Q_{CplxJ} (shown below) which joins tables $R_{\#grp}^{1k}$ and R_{join}^c and each tuple from $R_{\#grp}^{1k}$ has 10 join partners. We control the number of conjunctive predict blocks, $n \in \{1, 2, 3\}$, that can form a disjunction. The total provenance sizes are $5M \cdot P_{SF}$, $10M \cdot P_{SF}$ and $15M \cdot P_{SF}$ for n ranging from 1 up to 3.

```
SELECT ga, count(vb) AS cvb, avg(vb) AS avb
FROM R#grp1k, Rjoinc
WHERE
  (ga >= 0 AND ga <= 300
   AND va >= 20 AND va <= 280
   AND vb >= 150 and vb <= 250
   AND ga = c1to50)
OR
  (ga >= 301 AND ga <= 600
   AND va >= 320 AND va <= 580
   AND vb >= 450 AND vb <= 550)
```

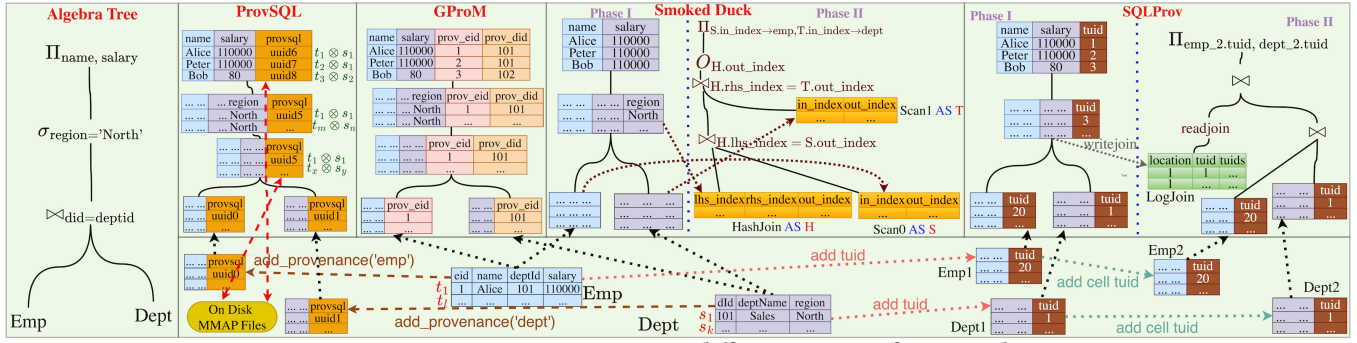


Figure 10: Comparing provenance capture in different systems for example query Q_{emp}

```

    AND ga = c1to50)
OR (ga >= 601 AND ga <= 900
AND va >= 620 AND va <= 880
AND vb >= 750 AND vb <= 850
AND ga = c1to50)
]
GROUP BY ga

```

C.2.2 Multi-Level Aggregation. T16: Multi-level Aggr. applies a multi-level aggregation query. Such queries challenge provenance systems like GProM that compute provenance bottom-up through propagation as it significantly increases the size of intermediate results. We use query Q_{MLAgg} (shown below) over a single table R_{tp}^{10M} , $10M \cdot P_{SF}$. We fix the total provenance size and control the number of groups $n \in \{1K \cdot P_{SF}, 1M \cdot P_{SF}\}$ in the inner aggregation by varying the group by attribute A_{grp} . Q_{MLAgg} first computes n aggregation tuples which are then further aggregated into a single row by the outer aggregation.

```

SELECT max(va) AS maxava, max(vb) AS maxvb
FROM (SELECT gb, avg(va) AS va, avg(vb) AS vb
      FROM  $R_{tp}^{10M}$  GROUP BY  $A_{grp}$ ) AS sub

```

C.2.3 Argmin. T17: Argmin is the argmin query Q_{Amin} (shown below) which is a one-to-ten join query that computes the minimum for a group and then joins the aggregation result back to the input table to retrieve additional attributes from the row(s) with the minimum value.

Pengyuan says: added below

Boris says: Is there anything interested to be said about the provenance structure?

This query runs over two tables, R_{grp}^n and R_{join}^c , and yields a factorized polynomial $(\sum_i r_i) \cdot s_i$. We vary the parameter $n \in \{100, 1K\}$ to vary the total provenance size in $\{\sim 100K, \sim 1M\}$ but fix the per tuple provenance size, 1K.

```

SELECT ga, minv, c1to10
FROM (SELECT ga, min(va) AS minv
      FROM  $R_{grp}^n$ 
      GROUP BY ga) AS R
JOIN  $R_{join}^c$  AS S ON R.minv = S.c1to10

```

C.2.4 Recursive. T18: Recursive Query uses a recursive query Q_{RCR} (shown below) that returns the end points and lengths of paths in a graph (relation R_{graph}) that have less than d steps. We vary parameter $d \in \{2, 3\}$.

Pengyuan deleted:delete

Boris says: Say something about how this affects result size by choosing d because added

The R_{graph} represents a dense random graph where each node has an average out degree of 100. Thus, Increasing the d will lead to an exponential expansion of search space. The size of the provenance for this query grows significantly in d .

```

WITH RECURSIVE rcr AS (
SELECT f, t, 1 AS d
FROM  $R_{graph}$ 
WHERE f <= 10 *  $P_{SF}$ 
UNION ALL
SELECT p.f AS f, p.t AS t, r.d + 1 AS d
FROM rcr as r,  $R_{graph}$  as p
WHERE r.t = p.f and r.d < d
)
SELECT * FROM rcr;

```

C.2.5 Window Function. T19: Window Func. evaluates provenance capture for queries containing window functions. Window functions can result in large provenance. For instance, when using windows with **UNBOUNDED PRECEDING** the provenance of a row contains all rows that precede this row in sort order. However, such provenance is highly compressible as the provenance of consecutive rows overlap to a large degree.

Boris deleted:In this part, we examine SQLProv, the system can work on window functions, behavior to capture provenance using window functions.because to experiments

We use three variants of query Q_{win} (shown below) accessing table R_{tp}^{10M} , size of $10M \cdot P_{SF}$: the first two queries use **PARTITION BY**, varying group by attribute A_{grp} to vary the number of groups in $\{10, 1k\}$, resulting in provenance of limited size per result row while the third omits the **PARTITION BY** clause, resulting in large provenance (on average half the input table is in the provenance of a result tuple).

```

SELECT ga, avg(va) OVER ([PARTITION BY  $A_{grp}$ ]
                        ORDER BY id) AS rkava
FROM  $R_{tp}^{10M}$ ;

```

C.2.6 Limit Operator. T20: Limit Op. evaluates provenance capture for queries having limit operator (non Top-k). Limit operator can examine if a provenance system exploits the optimization to short-circuit execution and terminate early at the moment it hits the limit boundary. And limit operator test a system with large

intermediate result but small final provenance size of query result. We use the query template Q_{Lmt} (shown below), applying **LIMIT** operation on top of group-by-aggregation, accessing a single table R_{fp}^{10M} . The total size of provenance is fix and the total number of tuples inputted into **LIMIT** is $P_{\text{SF}} \cdot 1M$. We vary the $P_{\text{surviving}} \in \{0.1\%, 1\%, 10\%\}$ by controlling the number of tuples, $n \in \{1K, 10K, 100K\}$, **LIMIT** restricts.

```
SELECT gc, avg(va) AS ava, avg(vc) AS avc
FROM Rfp10M
GROUP BY gc
LIMIT n
```

C.2.7 Set Operator. T21:Set Op. evaluates provenance capture for negative queries using set operator **EXCEPT**. We use the query template Q_{Set} (shown below) accessing a single table R_{fp}^{10M} . The total provenance size is fixed, $10M \cdot P_{\text{SF}}$ and the amount of data from both sides of **EXCEPT** is the same. We vary the data remove rate, r , of left side to right side of **EXCEPT** operator. For example, $r = 0.1$ indicates that 10% data from both sides are the same. The remove rate is controlled by varying value range, rng , in right side.

```
(SELECT gb FROM fp WHERE gb <= 500)
EXCEPT
(SELECT gb FROM fp WHERE rng)
```

Table 4 presents an overview of query templates discussed in Sec. C.

D ADDITIONAL EXPERIMENTAL RESULTS

D.1 Use-case Specific Tasks

T2: Forward Debugging. Fig. 5c and show the end-to-end and breakdowns evaluation for templates Q_{FD1} and Q_{FD2} , respectively. Note that SmokedDuck does not support for this kind of debugging since it uses the physical row id to trace provenance and there is a no general approach to find the connection of identifiers between the result tuples and provenance tuples.

T2: Forward Debugging. Fig. 11b shows show the end-to-end and breakdowns evaluation for templates Q_{FD1} and Q_{FD2} , respectively. Note that SmokedDuck does not support for this kind of debugging since it uses the physical row id to trace provenance and there is a no general approach to find the connection of identifiers between the result tuples and provenance tuples.

T13: Forward Debugging. For debugging, lineage serves as a tool to trace the error original data and query and result. The above debugging experiment focus on data or result solely, in this experiment, we focus on the combination of lineage and result together. When an error is detected in the result or in the original data, it is interested to know which lineage data cause what error result. Thus, mapping lineage to result is needed in this case. Fig. 11e and Fig. 11f present the evaluation for this use case Q_{FD1} and Q_{FD2} .

T4: Reproducibility. Fig. 11g show the experimental result of query Q_{Lin} for reproducibility using provenance models lineage, minimal witness and 20% lineage. These figures shows (1) the end-to-end performance of all systems, and (2) performance breakdowns of systems SQLProv, ProvSQL and SmokedDuck. ProvSQL needs to read from and write to memory mapped files, and thus this leads worst performance among all systems. The minimal witness model

requires least cost of time and storage to complete the reproducibility tasks, because the amount of total provenance data becomes much less compared to lineage model which has the worst performance. For SmokedDuck and SQLProv, the percentage of phase I over total capture time increases when size of lineage decreases, because (1) phase II is more sensitive to the size of materialized data, and less lineage means less computation in phase II, and (2) phase I is a must-have phase even if a query produce no results. In terms of storage usage, lineage model is obviously largest. Note that for minimal witness, SmokedDuck consumes storage but data size is less than one block, and thus, actual value cannot be queried. Minimal witness model is an optimized for reproducing the result (reproduce cost) in terms of both execution and storage cost.

T14: Reproducibility. Fig. 11h presents the GProM’s capability to capture the coarse-grained provenance. Compared to the table fp partitioned 10 ranges, number of ranges of 100 and 500 lead to higher cost in capture the provenance but less cost in reuse the sketches. Thus, the finer-grained partitioning leads to skip more irrelevant data to achieve faster query processing.

D.2 Provenance Structure & Size

D.3 Query Constructs & Structure

T15: Complex Join Conditions. Fig. 13b shows the evaluation of system to capture provenance for the query Q_{CplxJ} . All systems show a trend of increasing runtime when the join condition blocks increase. Since the join predicates prevent the systems from applying query optimizations, they must evaluate a large number of join conditions during query execution. As a result, provenance capture becomes increasing expensive when the number of join predicates grows.

Fig. 13c shows the experimental evaluation for Q_{MLAgg} . All systems show a trend of runtime increasing when there are more groups inside the inner aggregation, as it increases the total provenance size and the number of groups to be evaluated in the outer aggregation.

T17: Argmin. Fig. 13d presents the experimental evaluation of provenance capture for query Q_{AMin} . In this experiment, we fix the per result tuple provenance size P_{res} at 1K and vary the number of aggregation groups. Since the Q_{AMin} computes an aggregation first and then joins the aggregation result with join another table, increasing the number of groups directly increases the number of aggregation results, and thus, the number of tuples participating in join, leading to the increasing of total provenance size. GProM performs the worst as it cannot exploit the factorized provenance for queries.

T18: Recursive. Fig. 13e shows the provenance capturing performance for Q_{RCR} . The results show that the complexity of a path (i.e. the number of nodes in a path) determines the cost of performance. As the depth increases, provenance capture becomes more expensive because additional recursive iterations must be evaluated, which increase the provenance size and capture runtime.

T19: Window Function. Fig. 13f presents the evaluation for Q_{Win} . The results show that (1) introducing a **PARTITION BY** clause increases the provenance capture runtime, (2) the runtime further increases as the number of partitions grows.

Task	Query	Tables	Parameter	Join Format	Tot. Prov.	Inter. Res.	Res.
T12: Forward Debugging	Q_{FD1}	$R_{agg}^{1K}, R_{join}^{1K}$	–	1000%(1 – 10)	~ 1M	10M	1K
T13: Forward Debugging	Q_{FD1}	$R_{agg}^{1K}, R_{join}^{1K}$	–	1000%(1 – 10)	~ 0.33M	10M	333
	Q_{FD2}	$R_{agg}^{1K}, R_{join}^{1K}$	–	1000%(1 – 10)	~ 0.33M	10M	~ 3.3M
T14: Reproducibility	Q_{AP}	$R_{ip}^{10M}, R_{join}^{1K}$	–	1000%(1 – 10)	~ 10M	100M	15
T15: Complex Join Conditions	Q_{Cplx}	$R_{agg}^{1K}, R_{join}^{1K}$	$n \in \{1, 2, 3\}$	500%(1[10%] – 50), 1000%(1[20%] – 50), 1500%(1[30%] – 50)	~ 0.1M, ~ 0.2M, ~ 0.3M	~ 5M, ~ 10M, ~ 15M	{100, 200, 300}
T16: Multi-Level Aggregation	Q_{MLAgg}	R_{ip}^{10M}	$n \in \{1K, 1M\}$	–	~ 10M, ~ 15M	10M	{1K, 1M}
T17: Argmin	Q_{AMin}	$R_{agg}^{1K}, R_{join}^{1K}$	$n \in \{100, 1K\}$	1000%(1 – 10)	~ 0.1M, ~ 1M	{1M, 10M}	{1K, 10K}
T18: Recursive	Q_{RCR}	R_{graph}	$d \in \{2, 3\}$	–	1 K	–	~ 110K, ~ 1.1M
T19: Window Function	Q_{Win}	R_{ip}^{10M}	w/o partition, w/ partition(A_{grp})	–	10M	–	10M
T20: Limit Operator	Q_{Lmt}	R_{ip}^{10M}	$n \in \{1K, 10K, 100K\}$	–	10M	1M	{1K, 10K, 100K}
T21: Set Operator	Q_{Set}	R_{ip}^{10M}	$rng \in \{[401, 500], 250, 500\}, [1, 500]\}$	–	10M	–	{400, 250, 0}

Table 4: Query templates ($P_{SF} = 1$) discussed in Sec. C: tables accessed, varied parameters, join format, total provenance size (Tot. Prov.), intermediate result size (Inter. Res.) and result size (Res.)

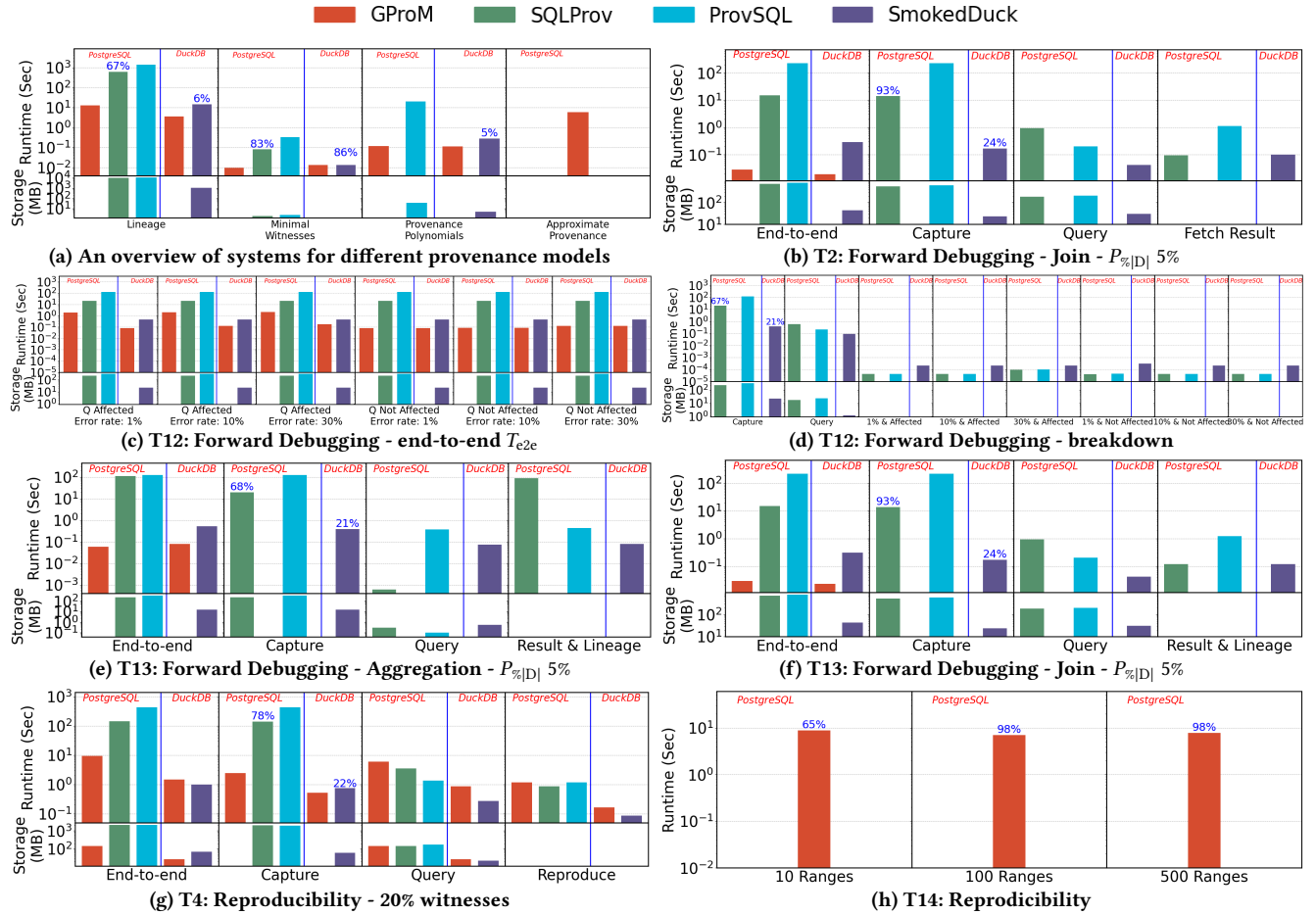


Figure 11: Use-case specific tasks: additional experiments

T20: Limit Operator.

Fig. 13g the results of provenance capture for query template Q_{Lmt} . ProvSQL and SQLProv benefit when the **LIMIT** predict is selective, while GProM and SmokedDuck keep the performance stable when varying the number of tuples returned by **LIMIT**.

T21: Set Operator.

E JOIN SELECTIVITY

We now discuss the selectivity of joins used in queries in the benchmark. Consider a star join query, where R_0 is the center of the star, i.e., R_0 joins with each R_i .

```

SELECT ...
FROM  $R_0, R_1, \dots, R_n$ 
WHERE ...

```

We use a unified notation for join query selectivity as *selectivity profiles* that compactly describe the selectivity in a query with

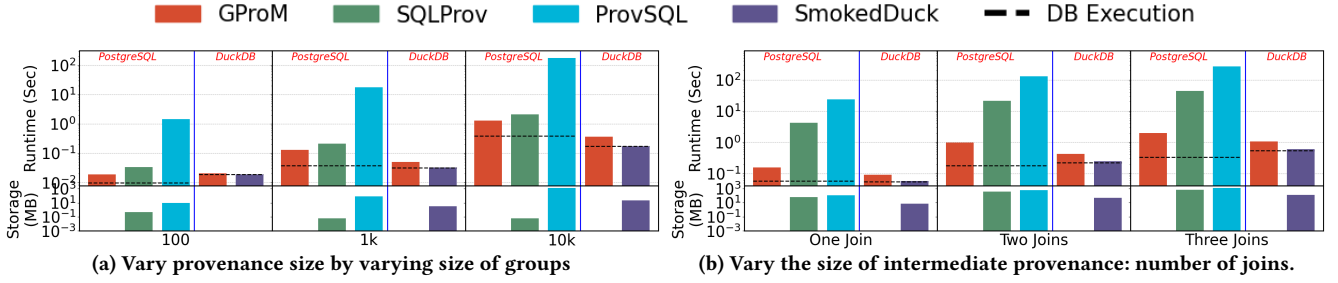


Figure 12: Provenance structure & size: additional experiments

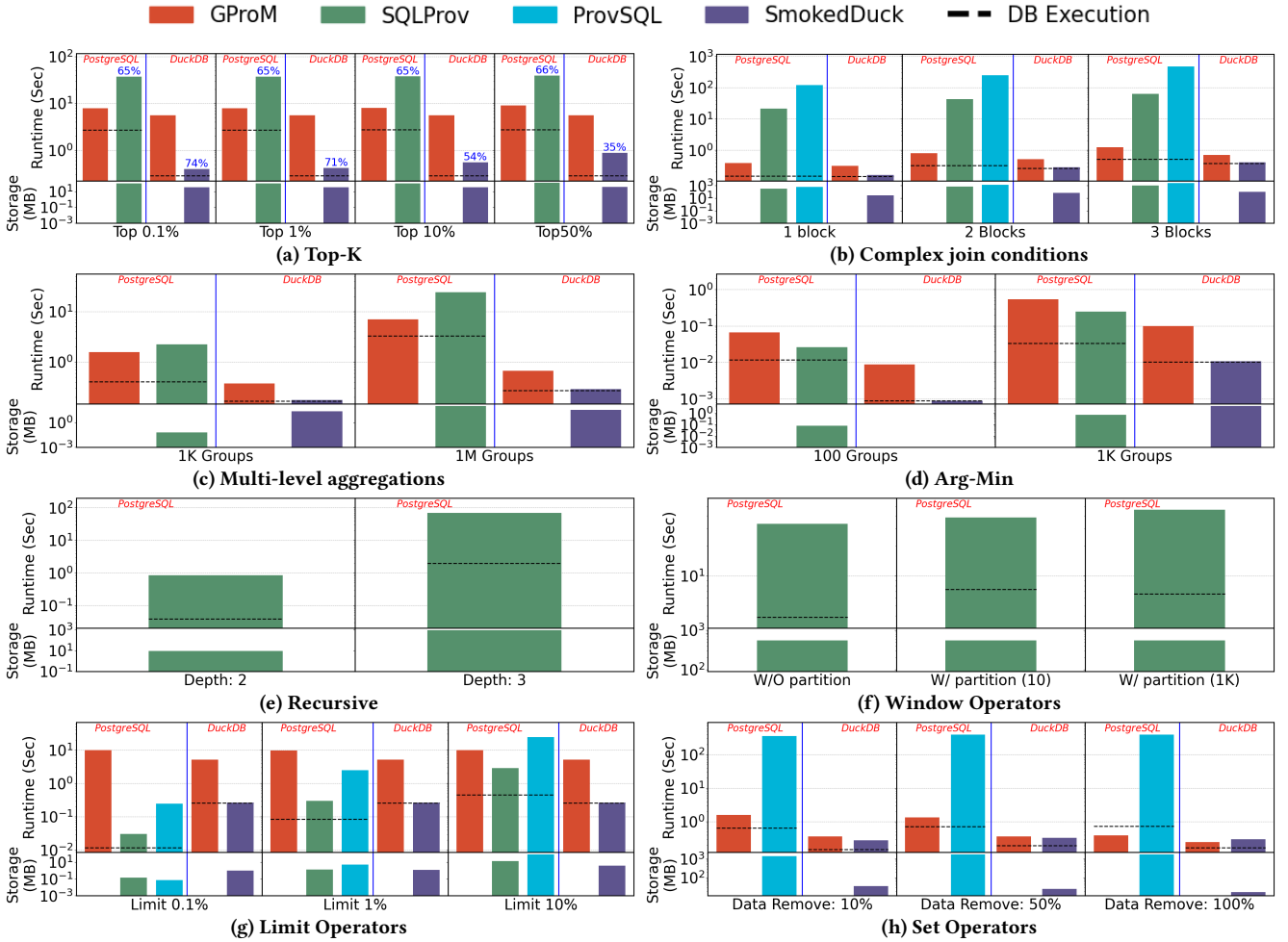


Figure 13: Query constructs & structure: additional experiments

multiple joins. The *selectivity profile* of such an n -way join is of the form $x(1[y] - j_1 - \dots - j_n)$. Here, $x\%$ is the join result size relative to the size of the left-most table R_0 , e.g., $x = 2.0$ would imply that the result of the join is twice the size of relation R_0 . Parameter $y\%$ is the fraction of tuples from the left-most table R_0 that have join partners with the result of joining all remaining tables. Furthermore,

j_i is the average number of tuples from the $i + 1^{th}$ table that can join with a tuple from R_0 . For example, $100\%(1[10\%] - 10)$, is a query with one join whose output is the same size as R_0 due roughly 10% of tuples from R_0 having join partners, but each such tuple on average has 10 join partners in R_1 . As another example, consider $500\%(1[10\%] - 10 - 5)$ which describes a three-way join query

which returns $5 \cdot |R_0|$ tuples, because on average every of the 10% tuples from R_0 that have join partners in both R_1 and R_2 do join with 10 tuples in R_1 and 5 tuples in R_2 .

E.1 Selectivity Profiles for Benchmark Queries

Boris says: @Pengyuan add here: the ones I copied here are from "Varying Intermediate Provenance Size"

$100\%(1[10\%] - 10)$ $500\%(1[10\%] - 10 - 5)$ $1000\%(1[10\%] - 10 - 5 - 2)$.